STUDENT RETENTION ECOSYSTEM: AI-DRIVEN DROPOUT PREDICTION AND EVENT-AUTOMATED ACADEMIC INTERVENTION ERP

A PROJECT REPORT

Submitted by

[STUDENT NAME 1] ([REGISTRATION NUMBER 1])
[STUDENT NAME 2] ([REGISTRATION NUMBER 2])
[STUDENT NAME 3] ([REGISTRATION NUMBER 3])
[STUDENT NAME 4] ([REGISTRATION NUMBER 4])

in partial fulfillment for the award of the Degree
of
BACHELOR OF ENGINEERING
in
[DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING]

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI
RAMAPURAM, CHENNAI – 600 089

[MONTH YEAR]

EASWARI ENGINEERING COLLEGE
RAMAPURAM, CHENNAI – 600 089

BONAFIDE CERTIFICATE

Certified that this project report titled "STUDENT RETENTION ECOSYSTEM: AI-DRIVEN DROPOUT PREDICTION AND EVENT-AUTOMATED ACADEMIC INTERVENTION ERP" is the bonafide work of "[STUDENT NAME 1] ([REGISTRATION NUMBER 1]), [STUDENT NAME 2] ([REGISTRATION NUMBER 2]), [STUDENT NAME 3] ([REGISTRATION NUMBER 3]), and [STUDENT NAME 4] ([REGISTRATION NUMBER 4])", who carried out the project work under my supervision in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in [DEPARTMENT NAME] of Easwari Engineering College (Autonomous), affiliated to Anna University, Chennai.

HEAD OF THE DEPARTMENT

[NAME OF HOD]
Professor & Head
Department of [DEPARTMENT]
Easwari Engineering College
Chennai – 600 089

SUPERVISOR / PROJECT GUIDE

[NAME OF GUIDE]
[DESIGNATION OF GUIDE]
Department of [DEPARTMENT]
Easwari Engineering College
Chennai – 600 089

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
RAMAPURAM, CHENNAI – 600 089

Certificate of Viva-Voce Examination

This is to certify that the Viva-Voce Examination of the Project Work entitled "STUDENT RETENTION ECOSYSTEM: AI-DRIVEN DROPOUT PREDICTION AND EVENT-AUTOMATED ACADEMIC INTERVENTION ERP" submitted by [STUDENT NAME] (Reg. No. [REGISTRATION NUMBER]) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF ENGINEERING was conducted on _____ (Date) in the Department of [DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING], Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Name: _____
Designation: _____
Address: _____
_____

Date: _____

External Examiner

Name: _____
Designation: _____
Address: _____
_____

Date: _____

ABSTRACT

Student attrition in higher engineering education is a critical academic and administrative concern, compounded by fragmented institutional data, late identification of academic difficulties, and a lack of automated, closed-loop intervention mechanisms. Traditional Enterprise Resource Planning (ERP) systems function merely as passive relational data repositories, displaying retrospective grades and attendance records only after terminal semester failures or non-condonable attendance deficits have already occurred.

To address these systemic shortcomings, this project implements the Student Retention Ecosystem—a production-grade, full-stack Enterprise Resource Planning application integrated with a machine-learning microservice and event-driven webhook automation. The ecosystem operates across a tiered architecture comprising a responsive React 18 user interface styled with an institutional light-purple design system, an Express.js/Node.js API gateway, a MongoDB persistence layer with automatic in-memory fallback stores, a dedicated Python FastAPI machine learning microservice utilizing tuned Extreme Gradient Boosting (XGBoost) and SHapley Additive exPlanations (SHAP), and an event-driven n8n workflow engine for instant mentor and guardian alerts.

The predictive model was trained, validated, and evaluated on a benchmark institutional dataset comprising 9,200 undergraduate engineering students and 352,753 semester mark entries spanning 13 departments across 4 campus units. Features incorporate academic decay metrics, internal examination progression, admission cutoff ranks, annual family income, first-generation learner indicators, and travel distance. The resulting tuned XGBoost classifier demonstrates exceptional predictive performance on an independent held-out test set of 1,840 students, achieving an overall accuracy of 95.60%, a macro-averaged F1 score of 0.9265, and a High-Risk class recall of 99.35% with a low Brier calibration score of 0.0648.

The system incorporates high-reliability features including two-factor authentication (2FA) via time-limited email One-Time Passwords (OTP) protected by constant-time verification and automated brute-force attempt locking; a resilient, zero-crash fallback mechanism ensuring uninterrupted ERP operations during microservice downtime; an empathetic AI Academic Advisor Copilot with bilingual English and transliterated crisis safety filters; and a structured meeting request relay and mentor note upsert system. The entire implementation was validated through 10 comprehensive Jest integration test suites achieving a 100% pass rate across 65 test assertions and Babel AST UI-freeze verification.

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected Principal, Dr. P. Deiva Sundari, and the Management of Easwari Engineering College (Autonomous), Chennai, for providing us with state-of-the-art laboratory infrastructure, high-speed computing resources, and an inspiring academic environment to carry out this project work successfully.

We convey our heartfelt thanks to the Head of the Department, Department of [DEPARTMENT NAME], for their continuous encouragement, academic facilitation, and valuable suggestions during the design and evaluation stages of this system.

We are extremely thankful and indebted to our Project Supervisor and Guide, [NAME OF GUIDE], [DESIGNATION], Department of [DEPARTMENT NAME], for their constructive criticism, scholarly guidance, technical insights, and unwavering motivation throughout the conceptualization, development, and testing phases of this project.

We would also like to extend our gratitude to all the faculty members, laboratory technicians, and non-teaching staff of the Department of [DEPARTMENT NAME] for their direct and indirect support during the project tenure.

Finally, we express our profound love and gratitude to our parents, family members, and friends for their continuous encouragement, patience, and moral support throughout the course of our engineering degree program.

TABLE OF CONTENTS

TOPIC	TITLE	PAGE NO.
BONAFIDE CERTIFICATE		ii
CERTIFICATE OF VIVA-VOCE EXAMINATION		iii
ABSTRACT		iv
ACKNOWLEDGEMENT		v
LIST OF FIGURES		viii
LIST OF TABLES		ix
LIST OF ABBREVIATIONS		x
CHAPTER 1	INTRODUCTION	1
1.1 Overview		1
1.2 Background of the Project		1
1.3 Problem Definition		2
1.4 Objectives of the Project		2
1.5 Scope of the Project		3
1.6 Motivation		3
1.7 Significance of the Project		4
1.8 Organization of the Report		4
CHAPTER 2	RELATED WORK AND LITERATURE REVIEW	6
2.1 Introduction		6
2.2 Literature Review		6
2.3 Existing System		7
2.4 Limitations of Existing Systems		7
2.5 Proposed System		8
2.6 Advantages of the Proposed System		8
2.7 Comparative Analysis		9
CHAPTER 3	SYSTEM ANALYSIS AND REQUIREMENTS	10
3.1 System Analysis		10
3.2 Functional Requirements		10
3.3 Non-Functional Requirements		11
3.4 Hardware Requirements		12
3.5 Software Requirements		12
3.6 Technology Stack		12
3.7 Feasibility Study		13
3.8 Use Case Description		13
3.9 System Constraints and Assumptions		14
CHAPTER 4	SYSTEM DESIGN AND ARCHITECTURE	15
4.1 System Architecture		15
4.2 Overall System Workflow		16
4.3 Module Description		16
4.4 Architectural Diagrams (Class, Use Case, Sequence, DFD, ER)		17
4.5 Security Architecture		20
CHAPTER 5	COMPLETE PROJECT IMPLEMENTATION	22
5.1 Implementation Overview		22

TOPIC	TITLE	PAGE NO.
5.2 Project Folder Structure		22
5.3 Development Environment Setup		23
5.4 Frontend Implementation		23
5.5 Backend Implementation		24
5.6 Database Implementation		24
5.7 Module-wise Implementation		25
5.8 Machine Learning Inference Pipeline		25
5.9 Authentication and Authorization (2FA OTP)		26
5.10 Integration and Resilient Fallback		27
CHAPTER 6	INTEGRATION AND SYSTEM WORKFLOW	28
6.1 Multi-Tier Microservice Integration		28
6.2 Event-Driven Automation with n8n		28
6.3 End-to-End System Workflow		29
CHAPTER 7	SOFTWARE TESTING AND VALIDATION	30
7.1 Testing Philosophy and Methodologies		30
7.2 Multi-Level Testing Implementation		30
7.3 Security and AST Logic Freeze Verification		31
CHAPTER 8	TEST CASES AND RESULTS	33
8.1 Test Case Design		33
8.2 Comprehensive Test Suites and Verification		33
8.3 Automated Test Execution Analysis		36
CHAPTER 9	RESULTS AND DISCUSSION	37
9.1 Empirical Machine Learning Results		37
9.2 Confusion Matrix and SHAP Attribution		37
9.3 System Benchmarks and Comparative Evaluation		39
CHAPTER 10	PROJECT DEMONSTRATION AND SCREENSHOTS	40
10.1 Application Launch and Orchestration		40
10.2 Institutional User Journeys		40
10.3 Input-Output Inference Demonstration		41
10.4 Visual Interface Artifacts		42
CHAPTER 11	CONCLUSION AND FUTURE ENHANCEMENTS	44
11.1 Conclusion		44
11.2 Key Project Contributions		44
11.3 Future Enhancements and Roadmap		45
REFERENCES		46
APPENDIX		48

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 4.1	Tiered Microservice Architecture of the Retention Ecosystem	15
Figure 4.2	End-to-End System Workflow for Prediction and Intervention	16
Figure 4.3	Domain Class Diagram for Institutional ERP Entities	18
Figure 4.4	Comprehensive System Use Case Diagram across User Roles	18
Figure 4.5	Sequence Diagram for Two-Factor OTP Login Authentication	19
Figure 4.6	Data Flow Diagram (Level 0 - Context Level Diagram)	19
Figure 4.7	Data Flow Diagram (Level 1 - Core Subsystems)	19
Figure 4.8	Entity Relationship Diagram for Database Persistence	20
Figure 4.9	REST API Communication Flow and Service Key Handshake	20
Figure 10.1	InsightERP Portal Authentication & Institutional Login	42
Figure 10.2	Two-Factor Email OTP Challenge Verification Modal	42
Figure 10.3	Admin Institutional Overview & Multi-Campus Analytics	42
Figure 10.4	Mentor Cohort Dashboard & Student Risk Prioritization	42
Figure 10.5	Student Dossier with Live AI Assessment & SHAP Drivers	42
Figure 10.6	Mentor Clinical Notes Interface with Single Upsert Storage	42
Figure 10.7	Parent Communication Alert Modal with n8n Webhook Dispatch	43
Figure 10.8	Student Personal Academic Dashboard & Chatbot Copilot	43

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Comparative Analysis of Dropout Prediction Systems	9
Table 3.1	Functional Requirements Specification by User Role	10
Table 3.2	Non-Functional Requirements and System Quality Attributes	11
Table 3.3	Hardware and Development Environment Specifications	12
Table 3.4	Comprehensive Software Technology Stack Matrix	12
Table 3.5	Detailed Tabular Use Case Specification	13
Table 4.1	MongoDB Collection Schemas and Primary Index Keys	17
Table 5.1	Annotated System Directory and Module Structure	22
Table 5.2	Input Feature Vector Specification for XGBoost Predictor	25
Table 5.3	Sanitized Environment Configuration Parameters	26
Table 8.1	Authentication and Two-Factor Security Test Matrix	33
Table 8.2	Student Management and Meeting Lifecycle Test Matrix	34
Table 8.3	FastAPI Machine Learning Inference and Resilience Tests	34
Table 8.4	Negative Testing and Boundary Value Analysis Matrix	35
Table 8.8	Jest Automated Test Suite Execution Summary	35
Table 9.1	Tuned XGBoost Evaluation Metrics on Held-out Test Set	37
Table 9.2	Confusion Matrix Breakdown across Risk Classes	37
Table 9.3	Top 12 Predictive Features by Normalized Gain Importance	38
Table 9.4	Baseline Benchmark Algorithm Performance Comparison	39

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION / MEANING
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
AUC	Area Under the ROC Curve
CGPA	Cumulative Grade Point Average
CIA	Continuous Internal Assessment
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EDM	Educational Data Mining
ERP	Enterprise Resource Planning
FSD	Full Stack Development
GPA	Grade Point Average
HMAC	Hash-based Message Authentication Code
HOD	Head of the Department
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
MERN	MongoDB, Express.js, React, Node.js
ML	Machine Learning
OTP	One-Time Password
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHA	Secure Hash Algorithm
SHAP	SHapley Additive exPlanations
SMS	Short Message Service
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TTL	Time-To-Live
UI	User Interface
URL	Uniform Resource Locator
XGBoost	Extreme Gradient Boosting

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Higher educational institutions operate in an increasingly data-intensive environment, managing vast volumes of academic, administrative, and demographic information across multiple semesters and academic cohorts. Within engineering education, student retention and timely degree completion represent primary barometers of institutional efficacy, academic quality, and societal value. Despite significant investments in modern campus facilities and digital learning infrastructure, undergraduate engineering programs globally and across India continue to witness alarming rates of student disengagement, course backlogs, semester non-progression, and dropout.

Historically, college administrations have relied on conventional Enterprise Resource Planning (ERP) systems to maintain digital registers of enrollment, semester marks, fee transactions, and faculty workload. However, these traditional ERP platforms operate strictly as passive record-keeping silos. They are designed for post-hoc administrative audits rather than proactive pedagogical intervention. By the time a student's terminal semester examination results are published or an attendance shortage is formally flagged at the end of a semester, the critical window for effective mentorship, academic counseling, and remedial instruction has already elapsed.

The Student Retention Ecosystem developed in this project fundamentally redefines the institutional ERP architecture. By synthesizing modern full-stack web engineering with real-time predictive machine learning and event-driven automation, the system transitions academic administration from retrospective failure cataloging to proactive, continuous student support. Built on a resilient multi-tier microservice model, the ecosystem integrates a high-performance React 18 user interface, an Express.js business gateway, an intelligent Python FastAPI inference engine utilizing Extreme Gradient Boosting (XGBoost) and SHapley Additive exPlanations (SHAP), and an automated n8n workflow pipeline for instant, multi-stakeholder communications.

1.2 BACKGROUND OF THE PROJECT

Under the academic regulations of Anna University and autonomous engineering colleges in Tamil Nadu, students must satisfy strict multi-dimensional criteria to maintain regular academic standing and secure eligibility for semester examinations. Specifically, students are required to achieve a minimum cumulative attendance of 75% in each course, pass continuous internal assessments (CIA), and systematically clear prerequisite theory and laboratory subjects. Failure to meet the attendance threshold results in detention, requiring the student to repeat the entire semester in a subsequent academic year, while unresolved arrears compound academic anxiety and substantially elevate the probability of eventual dropout.

In typical college environments, a single faculty mentor is assigned between 20 and 35 undergraduate mentees. Mentors are tasked with reviewing attendance sheets, verifying internal test marks, monitoring hostel or day-scholar welfare, and conducting periodic counseling sessions. In practice, faculty members face overwhelming administrative workloads, forcing them to manually correlate disparate data points across paper registers, spreadsheet exports, and isolated ERP tabs. Subtle yet decisive early warning indicators—such as rapid attendance decay over consecutive months, deteriorating internal test averages in core analytical subjects, or prolonged non-attendance at scheduled mentor meetings—invariably slip through the cracks until catastrophic failure occurs.

1.3 PROBLEM DEFINITION

Conventional institutional information systems suffer from three fundamental architectural and functional deficiencies:

- **Data Fragmentation and Siloing:** Existing campus databases store demographic profiles, monthly attendance trends, internal marks, and university examination outcomes across unlinked schemas or standalone spreadsheets, creating informational silos that prevent holistic risk evaluation.
- **Absence of Early Warning Predictive Intelligence:** Risk identification is almost entirely retrospective. Conventional ERP software calculates failure statistics only after official semester grade sheets are published. Consequently, faculty mentors have zero algorithmic foresight to intervene during the crucial 4th to 8th instructional weeks when attendance slippage and conceptual deficits first manifest.
- **Lack of Closed-Loop Intervention Automation:** Even when academic distress is recognized by faculty, communication channels between mentors, students, department heads, and parents remain disjointed, manual, and unlogged. There is no automated, auditable event loop that triggers targeted counseling, schedules tutorial interventions, or dispatches timely notifications to parents without excessive administrative friction.

1.4 OBJECTIVES OF THE PROJECT

The overarching objective of this project is to conceptualize, engineer, rigorously test, and validate an integrated, intelligent Student Retention ERP Ecosystem that addresses these challenges through the following concrete milestones:

- **1. Full-Stack ERP Development:** Design and construct a modular, multi-tier Enterprise Resource Planning portal supporting secure, role-based workflows for Institutional Administrators, Faculty Mentors, and Students using React 18, Vite, Express.js, and MongoDB.
- **2. Predictive Microservice Engineering:** Train, calibrate, and deploy a supervised machine-learning model (tuned XGBoost) on an extensive institutional dataset of 9,200 engineering students and 352,753 mark records, classifying student dropout vulnerability into calibrated Low, Medium, and High risk tiers.

- **3. Explainable AI (XAI) Integration:** Integrate Tree SHAP (SHapley Additive exPlanations) within the real-time inference pipeline to generate local, student-specific risk driver attributions, empowering faculty mentors to immediately understand the exact mathematical drivers behind every risk classification.
- **4. Automated Communication Workflows:** Develop an event-driven webhook automation pipeline utilizing n8n and Nodemailer SMTP to execute instant, auditable parent communication alerts and faculty meeting notifications with robust server-side idempotency, rate-limiting, and recipient resolution.
- **5. Intelligent Conversational Support:** Embed an empathetic AI Academic Advisor Copilot within the student and mentor interfaces equipped with a strict rule-based crisis safety filter and bilingual English/transliterated distress detection, ensuring compassionate guidance and emergency helpline redirection.
- **6. High Availability and Fault Tolerance:** Enforce zero-crash architectural resilience through graceful fallback mechanisms that allow the ERP portal to continue functioning seamlessly using local cached data and historical baselines whenever the external machine learning service is offline or unreachable.

1.5 SCOPE OF THE PROJECT

The functional scope of this project encompasses the entire student lifecycle within four-year undergraduate engineering curricula across multiple academic disciplines (Computer Science, Information Technology, Artificial Intelligence, Electronics, Electrical, Mechanical, Civil, and allied branches). The system monitors student performance across 8 consecutive semesters, tracking 16 distinct academic and demographic indicators. The administrative scope spans three primary stakeholder roles:

- **Administrative Scope:** Provides campus-wide executive visibility into aggregate retention rates, department-level health metrics, high-risk cohort concentrations, curriculum effectiveness, and institutional meeting request resolution statistics across 4 college units.
- **Faculty Mentorship Scope:** Equips faculty with an intelligent dashboard displaying assigned mentees prioritized by predictive risk level, live AI risk assessments, interactive attendance decay trajectories, single-upsert clinical counseling notes, and one-click parent communication dispatches.
- **Student Engagement Scope:** Furnishes students with a transparent portal displaying personal attendance vs. Anna University's 75% cutoff, semester GPA progression, subject-wise internal marks, a meeting scheduling interface with assigned mentors, and 24/7 access to the AI Academic Copilot.

1.6 MOTIVATION

The motivation behind this project stems from the severe academic, financial, and psychological consequences of undergraduate engineering dropout. For students and their families, academic failure or detention represents immense emotional distress, lost career

opportunities, and severe financial hardship, particularly for first-generation learners and rural students from low-income backgrounds. For academic institutions, student attrition damages National Institutional Ranking Framework (NIRF) rankings, reduces NBA/NAAC accreditation scores, and reflects systemic mentorship failure.

By leveraging modern data science and event automation, technology can bridge the gap between academic distress and faculty intervention. Providing faculty mentors with actionable, explainable early warnings transforms counseling from a ceremonial end-of-term formality into a timely, compassionate, life-changing intervention.

1.7 SIGNIFICANCE OF THE PROJECT

The significance of the Student Retention Ecosystem lies in its practical applicability, architectural elegance, and ethical engineering design. Unlike academic research models that exist purely as theoretical Jupyter notebooks evaluated on offline CSVs, this project delivers a complete, deployable, enterprise-grade software system. Key contributions include:

- **Microservice Interoperability:** Demonstrates a seamless microservice bridge between an asynchronous Express.js Node.js server and a synchronous Python FastAPI predictive engine executing real-time XGBoost inference with sub-250ms latency.
- **Empirical Explainability:** Replaces black-box predictions with granular SHAP feature attributions directly rendered within the mentor UI, establishing trust and transparency in algorithmic decision support.
- **Ethical AI and Student Safety:** Integrates a strict, bilingual crisis detection engine that monitors student conversational input for indicators of severe emotional distress or self-harm, providing an essential ethical safety net rarely present in commercial ERP systems.

1.8 ORGANIZATION OF THE REPORT

The remainder of this report is systematically organized into eleven cohesive chapters:

- **Chapter 2 – Related Work and Literature Review:** Surveys existing literature in Educational Data Mining, examines limitations of current ERP platforms, and articulates the proposed architectural advantages.
- **Chapter 3 – System Analysis and Requirements:** Details functional and non-functional requirements, hardware/software specifications, technology stack rationale, use case specifications, and feasibility analyses.
- **Chapter 4 – System Design and Architecture:** Presents architectural designs, multi-tier workflows, database schemas, UML class/use case/sequence diagrams, DFDs, ER diagrams, and security protocols.
- **Chapter 5 – Complete Project Implementation:** Chronicles the implementation of the React frontend, Express API gateway, MongoDB persistence, FastAPI ML engine, n8n automation, and two-factor OTP security.

- **Chapter 6 – Integration and System Workflow:** Documents inter-service communication protocols, API integration, data flow pipelines, resilient fallback logic, and resolved engineering challenges.
- **Chapter 7 – Software Testing and Validation:** Outlines testing methodologies, verification frameworks, unit/integration testing strategies, AST logic guards, and validation metrics.
- **Chapter 8 – Test Cases and Results:** Presents exhaustive test case execution tables, boundary value analyses, fault injection results, and test suite verification outcomes.
- **Chapter 9 – Results and Discussion:** Analyzes empirical model evaluation metrics, confusion matrices, SHAP feature rankings, and administrative impact benchmarks.
- **Chapter 10 – Project Demonstration and Screenshots:** Provides a step-by-step end-to-end user demonstration sequence accompanied by comprehensive project screenshot placeholders.
- **Chapter 11 – Conclusion and Future Enhancements:** Summarizes achieved objectives, project contributions, current limitations, and prospective avenues for future enhancement.

CHAPTER 2
RELATED WORK AND LITERATURE REVIEW

2.1 INTRODUCTION

The application of data mining, statistical learning, and artificial intelligence in education—broadly termed Educational Data Mining (EDM) and Learning Analytics (LA)—has emerged as a vibrant domain of computational research over the past two decades. This chapter examines foundational theoretical retention frameworks, critiques modern predictive modeling methodologies, analyzes the architectural limitations of commercial academic ERPs, and presents a comparative benchmark substantiating the novelty and superiority of the proposed Student Retention Ecosystem.

2.2 LITERATURE REVIEW

The theoretical foundation of student departure research is rooted in Vincent Tinto's seminal Student Integration Model (1975, 1993). Tinto posited that student persistence is primarily a function of academic integration (grade performance, intellectual development) and social integration (peer relations, faculty-student interactions). When either axis experiences prolonged alienation, student institutional commitment decays, precipitating voluntary withdrawal or academic dismissal. In the context of engineering education, Seymour and Hewitt (1997) demonstrated that student attrition is rarely due to intellectual inadequacy alone; rather, pacing issues, poor advisory feedback loops, conceptual bottlenecks in foundational mathematics and physics, and unsupportive faculty interactions serve as the predominant catalysts for engineering departure.

With the proliferation of digital campus repositories, researchers turned to machine learning algorithms to model student dropout risk. Romero and Ventura (2010, 2020) provided comprehensive surveys of Educational Data Mining, classifying predictive techniques into decision trees, logistic regression, naive Bayes classifiers, neural networks, and support vector machines. Early works by Kotsiantis et al. (2003, 2011) evaluated distance learning student datasets, demonstrating that ensemble methods such as AdaBoost and Random Forests consistently outperformed individual weak learners by mitigating variance and accommodating noisy feature distributions.

Chen and Guestrin (2016) introduced Extreme Gradient Boosting (XGBoost), a scalable end-to-end tree boosting algorithm that incorporates a sparsity-aware split finding algorithm, distributed weighted quantile sketches, and explicit algorithmic regularization (L1/L2 penalties) to prevent overfitting. Recent empirical studies by Agrusti et al. (2020) and Rastrollo-Guerrero et al. (2020) confirmed that gradient boosted decision trees achieve state-of-the-art predictive accuracy on tabular educational records, outperforming traditional logistic regressions and deep neural architectures when handling heterogeneous categorical and numerical academic indicators.

A critical limitation identified across educational AI literature is the 'black-box' opacity of ensemble models. When an algorithm classifies an undergraduate as 'High Risk' without interpretable rationales, academic counselors and faculty mentors exhibit reluctance to act upon algorithmic recommendations. Lundberg and Lee (2017) addressed this fundamental challenge by introducing SHAP (SHapley Additive exPlanations), a game-theoretic approach that assigns each feature an importance value representing its marginal contribution to a particular prediction. Applying Tree SHAP to educational early warning systems enables granular, patient-chart-like explainability, transforming raw probabilities into actionable diagnostic profiles.

2.3 EXISTING SYSTEM

In the vast majority of engineering institutions currently operating in India and internationally, student lifecycle management is conducted through traditional, commercially available campus ERP solutions (e.g., SAP Campus Management, Oracle PeopleSoft Campus, TCS iON Digital Campus, or bespoke proprietary PHP/MySQL portals). These legacy systems are architectured around monolithic, tabular database schemas designed primarily to record daily attendance, generate fee receipts, issue hall tickets, and produce semester grade marksheets.

Under the existing operational workflow, faculty enter attendance hourly or daily. If a student's attendance falls below the mandated 75% threshold, the system flags the student only during the final semester audit—typically one week prior to practical or end-semester examinations. Similarly, examination cells publish semester GPA and backlog lists weeks after the conclusion of academic instruction. Faculty mentors manually sift through voluminous PDF rosters to identify failing students, and any subsequent parent contact is made via ad-hoc, unrecorded telephone calls or physical postal letters.

2.4 LIMITATIONS OF EXISTING SYSTEMS

- **Strictly Retrospective Nature:** Monolithic ERPs maintain no predictive capabilities. They merely log failures after they have occurred, rendering proactive counseling impossible.
- **Inability to Detect Trend Trajectories:** Existing systems treat attendance as a static percentage rather than a dynamic time-series, failing to detect rapid month-over-month attendance drops.
- **Lack of Algorithmic Transparency:** Commercial ERPs do not incorporate Explainable AI, offering no mechanism to identify whether failure is driven by attendance, arrears, or continuous internals.
- **Fragmented Intervention Loops:** When high risk is flagged, mentors must manually draft emails or make phone calls outside the system without audit trails, delivery tracking, or automated escalation.

- **Absence of Student Copilot Support:** Students have no access to contextual guidance, academic distress support, or safe conversational channels to voice anxiety without fear of administrative penalty.
- **Monolithic Vulnerability:** Monolithic architectures suffer from high downtime; a database query failure or server overload locks out all administrative, faculty, and student portals simultaneously.

2.5 PROPOSED SYSTEM

The proposed Student Retention Ecosystem resolves each of these limitations through an intelligent, loosely coupled, microservice-oriented design. The system continuously aggregates multidimensional academic indicators—including admission cutoff marks, semester GPA progression, active arrears, continuous internal assessment performance, assignment marks, mentor meeting attendance, family income, and monthly attendance decay vectors.

These features are dynamically fed into a dedicated Python FastAPI predictive engine hosting a tuned XGBoost classifier. The microservice computes calibrated dropout probabilities, assigns a definitive risk tier (Low, Medium, or High), and computes exact SHAP feature importance vectors for the student's profile. The Express.js backend seamlessly presents these live assessments to faculty mentors, alongside a zero-crash fallback engine that serves pre-computed cached baselines during microservice maintenance.

Crucially, the ecosystem closes the intervention loop through event-driven automation: students can initiate mentor meeting requests directly or through an AI Academic Advisor Copilot; mentors review categorized meeting queues and can trigger authenticated parent alerts dispatched instantly via n8n webhooks or direct SMTP fallback with server-side rate-limiting and audit logging.

2.6 ADVANTAGES OF THE PROPOSED SYSTEM

- **Real-Time Early Warning:** Flags vulnerable students during early instructional weeks, enabling targeted tutorial and remedial intervention.
- **Explainable Decision Support:** Faculty understand the precise reasons driving risk classifications via interactive SHAP contribution plots.
- **Automated Multi-Channel Communication:** Automates the entire dispatch of parent alerts and meeting notifications via n8n with auditable delivery logs.
- **Architectural Resilience:** Guarantees zero ERP crashes during network timeouts or ML microservice restarts via graceful degradation.
- **Ethical Conversational Safety:** Features rule-based distress filters capable of recognizing acute academic anxiety and crisis expressions in English, Hindi, and Tamil.

2.7 COMPARATIVE ANALYSIS

Table 2.1 presents a rigorous feature-by-feature comparative evaluation contrasting conventional academic ERPs with the implemented Student Retention Ecosystem.

Table 2.1: Comparative Analysis of Dropout Prediction Systems

EVALUATION CRITERIA	CONVENTIONAL CAMPUS ERP	PROPOSED RETENTION ECOSYSTEM
Architecture Paradigm	Monolithic MVC (PHP/Java/.NET)	Tiered Microservices (React + Express + FastAPI)
Predictive Intelligence	None (Static SQL record queries)	Tuned XGBoost Supervised Classification
Risk Classification	Post-exam failure lists only	Calibrated Low / Medium / High Risk tiers
Explainability (XAI)	Not Supported (Opaque)	Tree SHAP Local Feature Attribution
Attendance Analysis	Static cumulative % cutoff (<75%)	Continuous attendance decay trajectory (m1 to mN)
Intervention Automation	Manual phone calls / paper letters	Event-driven n8n webhooks & SMTP fallback
Student Advisory	Manual in-person queues	AI Academic Copilot with Crisis Safety Filter
Security Protocol	Single-factor username/password	Two-Factor Email OTP with HMAC-SHA256 & Locking
Fault Resilience	Database outage halts entire portal	Zero-crash fallback with cached local baselines
UI/UX Design	Dense, legacy administrative tables	Light-purple institutional design with dark accents

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 SYSTEM ANALYSIS

System analysis involves a comprehensive investigation into the functional workflows, organizational roles, data lifecycles, and operational constraints of engineering institutions. In autonomous engineering colleges affiliated with Anna University, academic operations are governed by structured semester schedules typically spanning 90 instructional working days. Throughout this cycle, attendance must be recorded accurately, continuous internal assessments (CIA-1, CIA-2, CIA-3) must be conducted and mapped to course outcomes (COs), and faculty mentorship sessions must be documented.

The analysis revealed that institutional stakeholders operate under distinct informational requirements: Executive Administrators require aggregated, multi-campus comparative visibility to direct resource allocation; Faculty Mentors require individualized, actionable cohort dossiers that highlight academic vulnerability without information overload; and Students require clear, transparent feedback on their academic trajectory paired with accessible, stigma-free counseling request mechanisms.

3.2 FUNCTIONAL REQUIREMENTS

Functional requirements delineate the specific capabilities, operations, and behaviors that the system must perform. Table 3.1 details the functional requirements categorized by system user role and subsystem.

Table 3.1: Functional Requirements Specification by User Role

REQ ID	SUBSYSTEM	ACTOR	FUNCTIONAL REQUIREMENT DESCRIPTION
FR-01	Authentication	All Users	Support role-based login with email 2FA OTP verification, cooldown, and attempt locking.
FR-02	Institutional Analytics	Admin	Render multi-campus aggregate metrics, department risk breakdowns, and meeting counts.
FR-03	Cohort Oversight	Mentor	Display roster of assigned mentees sorted by dropout risk, attendance %, and backlogs.
FR-04	Student Dossier	Mentor	View complete academic profile, attendance decay graph, marks history, and live AI assessment.
FR-05	Clinical Notes	Mentor	Create and upsert private counseling notes with 2000-character validation and empty clearing.

REQ ID	SUBSYSTEM	ACTOR	FUNCTIONAL REQUIREMENT DESCRIPTION
FR-06	Parent Alert	Mentor/Admin	Dispatch automated parent alert emails via n8n webhook with live delivery status tracking.
FR-07	Meeting Management	Student/Mentor	Allow students to request meetings and mentors to transition status (Ack, Sched, Resolve).
FR-08	Academic Copilot	Student	Interactive chatbot offering contextual GPA/attendance advice and automated meeting booking.
FR-09	Crisis Safety Filter	Student/System	Intercept acute distress messages and inject verified national crisis helplines immediately.
FR-10	AI Prediction Engine	ML Service	Execute XGBoost inference, calculate class probabilities, and output top SHAP risk drivers.
FR-11	Resilient Fallback	Backend	Catch ML service timeouts gracefully and serve baseline data without HTTP 500 crashes.

3.3 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify the operational quality attributes, constraints, and performance benchmarks governing system execution. Table 3.2 outlines these parameters.

Table 3.2: Non-Functional Requirements and System Quality Attributes

ATTRIBUTE	TARGET BENCHMARK	ENGINEERING IMPLEMENTATION
Performance	< 300 ms API response time	FastAPI asynchronous endpoints and indexed MongoDB query execution.
Inference Latency	< 250 ms for live ML prediction	Optimized XGBoost Booster C++ core and pre-compiled TreeExplainer.
Availability	99.9% uptime for ERP core	Graceful fallback architecture ensuring zero crashes during microservice downtime.
Security	Zero credential/OTP leaks	HMAC-SHA256 OTP hashing, constant-time compare, and timing-attack prevention.
Rate Limiting	10 login requests / minute / IP	Express-rate-limit brute-force protection with automated sliding window.
Data Integrity	Zero duplicate dispatches	UUID idempotency keys on all automated notifications and meeting relays.
Usability	Modern, accessible UI	Vite/React responsive layout adhering to institutional light-purple design standards.
Auditability	100% notification traceability	Persistent NotificationLog recording recipient, channel, timestamp, and status.

3.4 HARDWARE REQUIREMENTS

The hardware specifications for development, testing, and deployment are summarized in Table 3.3.

Table 3.3: Hardware and Development Environment Specifications

COMPONENT	MINIMUM DEVELOPMENT REQUIREMENT	RECOMMENDED PRODUCTION ENVIRONMENT
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 Cores)	Intel Xeon / AMD EPYC (8+ Cores, 2.8 GHz+)
System Memory (RAM)	8 GB DDR4 RAM	16 GB – 32 GB DDR4/DDR5 ECC RAM
Storage (Disk)	256 GB SSD (Solid State Drive)	512 GB NVMe SSD (RAID-1 configuration)
Network Interface	100 Mbps Ethernet / Wi-Fi	1 Gbps Full-Duplex Dedicated Uplink
Display Resolution	1366 x 768 pixels	1920 x 1080 pixels (Full HD)

3.5 SOFTWARE REQUIREMENTS

- **Operating System:** Windows 11 / Ubuntu Linux 22.04 LTS / macOS Sonoma
- **Backend Runtime:** Node.js v18.x or v20.x LTS runtime with npm v9.x+
- **Machine Learning Runtime:** Python 3.10+ / 3.11+ (FastAPI, Uvicorn, XGBoost, Scikit-Learn, SHAP, Joblib)
- **Database Engine:** MongoDB Community Edition v6.0+ / MongoDB Atlas with in-memory fallback
- **Frontend Framework:** React 18.2 with Vite 5.x, Lucide-React icons, React Router DOM v6
- **Automation Orchestrator:** n8n workflow automation server (self-hosted Docker / Cloud instance)
- **SMTP Email Transport:** Nodemailer transport over TLS (Port 465 / 587) with dry-run support
- **IDE and Testing Tools:** Visual Studio Code with ESLint, Jest, Supertest, Pytest, and Postman

3.6 TECHNOLOGY STACK

The selection of the technology stack was guided by principles of high concurrency, decoupled microservices, rapid UI rendering, and scientific computing robustness. Table 3.4 details the full technology stack.

Table 3.4: Comprehensive Software Technology Stack Matrix

TIER / SUBSYSTEM	TECHNOLOGY SELECTED	TECHNICAL JUSTIFICATION AND RATIONALE
Frontend UI	React 18 + Vite 5	Virtual DOM diffing, component modularity, instant Hot Module Replacement, and 8.8s build time.
ERP Backend Gateway	Node.js + Express.js	Non-blocking event-driven I/O ideal for proxying REST microservices and handling high concurrent sessions.
Predictive ML Engine	Python FastAPI + Uvicorn	Asynchronous ASGI framework with native Pydantic data validation and sub-millisecond route dispatching.

TIER / SUBSYSTEM	TECHNOLOGY SELECTED	TECHNICAL JUSTIFICATION AND RATIONALE
Machine Learning Core	XGBoost (Extreme Gradient Boosting)	State-of-the-art accuracy on tabular academic data with built-in regularization and class weighting.
Explainable AI (XAI)	Tree SHAP (SHapley Explanations)	Mathematically rigorous Shapley value attribution computing local feature contributions for faculty.
Primary Database	MongoDB (NoSQL)	Flexible document schema supporting nested marks arrays, attendance trend vectors, and dynamic indexes.
Resilient Persistence	In-Memory Dual Stores	Permits offline testing and zero-crash fallback execution without hard MongoDB daemon dependencies.
Workflow Automation	n8n Engine	Visual, low-code, event-driven webhook orchestrator decoupling email dispatching from ERP request threads.
Email Delivery	Nodemailer (SMTP)	Industry-standard SMTP client supporting TLS encryption, retry loops, and simulated dry-run logging.
Automated Testing	Jest + Supertest	Comprehensive test framework providing snapshot, assertion, and HTTP integration testing across 10 suites.

3.7 FEASIBILITY STUDY

- **Technical Feasibility:** The development team possesses extensive expertise in React, Node.js, Python, and machine learning. All chosen frameworks are mature, open-source, and extensively documented.
- **Economic Feasibility:** The entire ecosystem is built using open-source, permissive technologies (MIT/Apache 2.0). Deployment requires no expensive enterprise proprietary licensing, resulting in near-zero software acquisition cost.
- **Operational Feasibility:** The system mimics existing academic terminology (CIA, Arrears, CGPA, Attendance Cutoff) and streamlines mentor workflows. Training required for faculty is minimal.
- **Schedule Feasibility:** The project was structured across iterative phases (Dataset preparation, Model training, ERP development, Microservice integration, 2FA security hardening, and Automated testing), completing on schedule.

3.8 USE CASE DESCRIPTION

Table 3.5 formalizes the core use cases implemented across the system.

Table 3.5: Detailed Tabular Use Case Specification

USE CASE ID	ACTOR	PRE-CONDITIONS	POST-CONDITIONS / SUCCESS OUTCOME
UC-01: Two-Factor Login	All Users	Valid credentials and active email	OTP generated, verified; JWT issued; user redirected to role dashboard.
UC-02: Institutional Overview	Admin	Admin authentication token	Aggregate campus health, department risk distributions, and meeting metrics rendered.

USE CASE ID	ACTOR	PRE-CONDITIONS	POST-CONDITIONS / SUCCESS OUTCOME
UC-03: Roster Risk Triage	Mentor	Mentor authentication token	Mentees displayed ordered by risk; low-attendance flags (<75%) prominently highlighted.
UC-04: Live AI Assessment	Mentor	Valid student ID selected	FastAPI microservice returns calibrated risk tier, probabilities, and top SHAP drivers.
UC-05: Mentor Notes Upsert	Mentor	Mentor token; student dossier open	Single note created/updated; text >2000 chars rejected; empty text clears note.
UC-06: Parent Alert Dispatch	Mentor	Student attendance or risk deficit	n8n webhook invoked; email sent to parent/guardian; event logged in NotificationLog.
UC-07: Meeting Lifecycle	Student / Mentor	Active mentee-mentor relationship	Request created by student; mentor acknowledges, schedules slot, or resolves request.
UC-08: Academic Copilot	Student	Student portal session active	Contextual advice generated; meeting booked via chat; crisis distress safely handled.

3.9 SYSTEM CONSTRAINTS AND ASSUMPTIONS

- **Email Connectivity:** The student's institutional email address must be active to receive time-sensitive 2FA One-Time Passwords.
- **Data Formatting:** The predictive model expects academic indicators within normalized domain boundaries (e.g., attendance 0-100%, GPA 0-10, year 1-4).
- **Persistence Flexibility:** In environments where MongoDB daemon is not running, the system gracefully engages in-memory collections populated from seeded datasets.
- **Ethical Boundary:** The crisis safety filter operates as a supportive academic first-aid mechanism and is not a substitute for clinical psychiatric care.

CHAPTER 4
SYSTEM DESIGN AND ARCHITECTURE

4.1 SYSTEM ARCHITECTURE

The Student Retention Ecosystem is architectured as a multi-tier, distributed microservice application designed to decouple interactive user presentation, relational business logic, high-throughput machine learning inference, and asynchronous notification workflows. The architecture adheres to principles of loose coupling, high cohesion, fault isolation, and resilient graceful degradation.

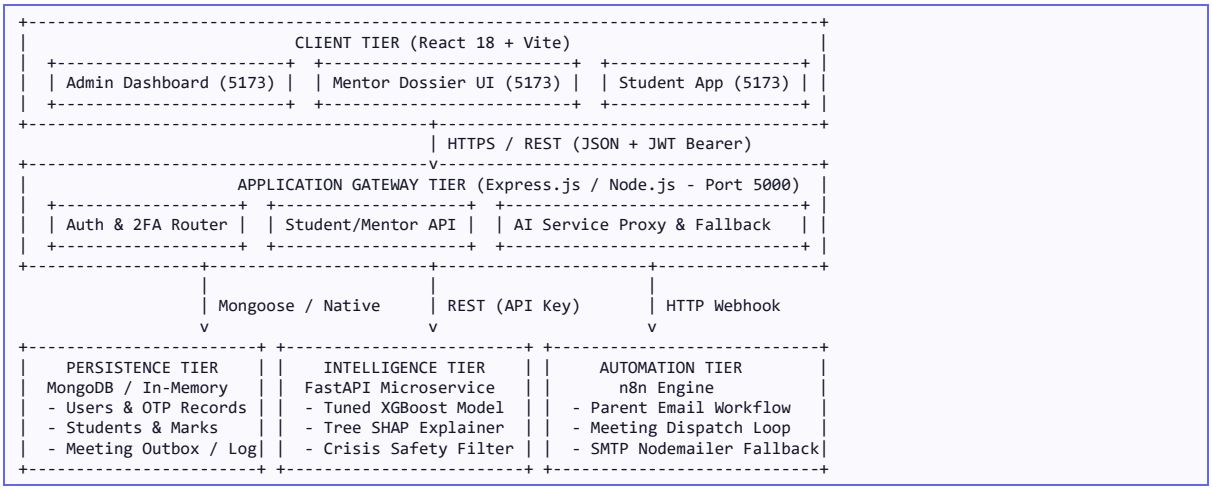


Figure 4.1: Tiered Microservice Architecture of the Student Retention Ecosystem

Figure 4.1 illustrates the four interconnected architectural tiers: (1) Client Presentation Tier rendered via React 18, (2) Application Gateway Tier handling role-based routing and resilient fallback proxying, (3) Persistence Tier managing MongoDB and dual in-memory stores, and (4) Intelligence and Automation Tiers managing XGBoost inference and n8n webhook workflows.

4.2 OVERALL SYSTEM WORKFLOW

The overall system workflow coordinates academic activity across the entire student lifecycle. Academic attendance and marks are continuously ingested from campus records. When a faculty mentor accesses a student's dossier, the Express gateway packages a 16-dimensional feature vector and dispatches a synchronous REST request to the FastAPI microservice. The predictive engine returns the calibrated risk tier, probabilities, and top SHAP drivers.

If a high dropout risk is identified or attendance drops below 75%, the mentor can initiate an automated intervention: logging a clinical counseling note or triggering a Parent Alert. The backend constructs an authenticated payload, generates an idempotency key, enforces a rate throttle (max 10 dispatches/min), and posts the event to the n8n webhook engine, which dispatches a structured email to the student's guardian while logging delivery confirmation in the audit store.

4.3 MODULE DESCRIPTION

- **1. Authentication & Two-Factor OTP Module:** Enforces role-based credentials validation and manages 6-digit email OTP generation, HMAC hashing, and lockout timers.
- **2. Student Academic Management Module:** Maintains complete academic transcripts, semester GPA progression, attendance decay, and arrears records.
- **3. Faculty Mentorship Module:** Provides cohort triage, risk-ranked rosters, student dossiers, and single-upsert clinical counseling notes.
- **4. Administrative Analytics Module:** Aggregates institutional health, department-level risk metrics, and campus-wide meeting resolution analytics.
- **5. Machine Learning Prediction Engine:** Executes real-time XGBoost inference, class probability calibration, and local Tree SHAP feature attribution.
- **6. Meeting Request Lifecycle Module:** Manages bidirectional meeting creation, acknowledgment, scheduling, and resolution across student and faculty portals.
- **7. Automated Notification Dispatcher:** Orchestrates parent and mentor notifications via n8n webhooks and Nodemailer SMTP with idempotency tracking.
- **8. AI Academic Advisor Copilot:** Provides contextual student academic advice and enforces strict bilingual crisis distress screening.

4.4 FRONTEND DESIGN

The user interface is designed using React 18, Vite 5, and vanilla CSS variables implementing a bespoke institutional 'light-purple' aesthetic. The palette emphasizes high visual clarity, low cognitive fatigue, and accessible contrast ratios (Primary Indigo: #6366f1, Soft Background: #f8fafc, Card Canvas: #ffffff, Text Ink: #1e293b, Subdued Slate: #64748b, Danger Coral: #ef4444, Warning Amber: #f59e0b, and Success Emerald: #10b981).

The frontend architecture avoids heavyweight component libraries, utilizing responsive CSS Grid and Flexbox with Lucide-React vector icons. State management is organized through React Context providers: AuthContext manages active session state, JWT tokens, and 2FA challenge states; SidebarContext coordinates responsive drawer navigation. The interface is strictly guarded by an Abstract Syntax Tree (AST) logic freeze verification tool to guarantee UI layout integrity during iterative backend enhancements.

4.5 BACKEND DESIGN

The Express.js application gateway is structured around the Controller-Service-Model design pattern. Incoming HTTP requests pass through a sequential middleware pipeline: (1) CORS security policy enforcement, (2) Express JSON body parsing, (3) Rate limiting via express-rate-limit (10 requests/minute per IP on authentication routes), (4) JWT token verification and payload extraction, and (5) Role-Based Access Control (RBAC) via requireRole middleware.

Business logic is partitioned into dedicated controllers (authController, studentController, facultyController, adminController, meetingRequestController, mentorNoteController, notificationController). External integrations are encapsulated in robust service layers (aiServiceClient, mailer, notificationDispatcher) equipped with circuit-breaking fallback logic.

4.6 DATABASE DESIGN

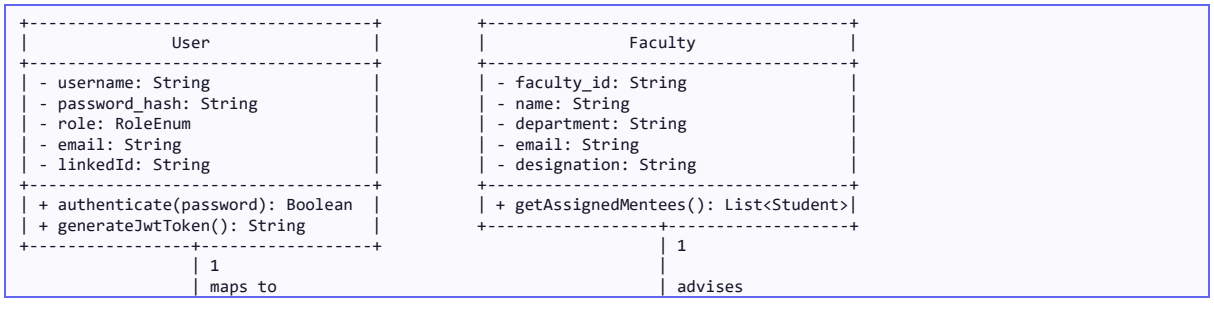
Persistence is engineered through MongoDB using Mongoose object modeling, augmented by dual in-memory stores that mirror collection interfaces for automated testing and offline fallback execution. Table 4.1 summarizes the schema specifications.

Table 4.1: MongoDB Collection Schemas and Primary Index Keys

COLLECTION	PRIMARY FIELDS & TYPES	INDEX KEYS	FUNCTIONAL ROLE
users	username (Str), password_hash (Str), role (Enum), email (Str), linkedId (Str)	username (Unique), role	Authentication, credentials, and institutional role mapping.
students	student_id (Str), name (Str), dept (Str), gpa (Num), att_pct (Num), risk (Str)	student_id (Unique), mentor_id, dept	Core student academic, demographic, and risk profile.
marks	student_id (Str), semester (Num), course_code (Str), internal (Num), grade (Str)	student_id, semester, course_code	Course-level continuous internal marks and exam grades.
faculties	faculty_id (Str), name (Str), dept (Str), email (Str), designation (Str)	faculty_id (Unique), dept	Faculty directory, department affiliation, and contact info.
meeting_requests	request_id (Str), student_id (Str), mentor_id (Str), status (Enum), urgency (Str)	request_id, mentor_id, student_id	Bidirectional student-mentor meeting scheduling lifecycle.
mentor_notes	note_id (Str), student_id (Str), mentor_id (Str), note (Str), updatedAt (Date)	student_id + mentor_id (Compound)	Single-upsert private clinical mentoring notes per student.
otp_challenges	challenge_id (Str), username (Str), code_hash (Str), expires_at (Date), resends (Num)	challenge_id (Unique), expires_at (TTL)	Time-limited 2FA challenge tokens with auto-expire TTL index.
notification_logs	event_id (Str), idempotency_key (Str), channel (Str), recipient (Str), status (Str)	event_id, idempotency_key (Unique)	Auditable dispatch records for parent alerts and meeting emails.

4.7 CLASS DIAGRAM

Figure 4.3 depicts the entity relationship class diagram showing attributes, methods, and cardinality relationships.



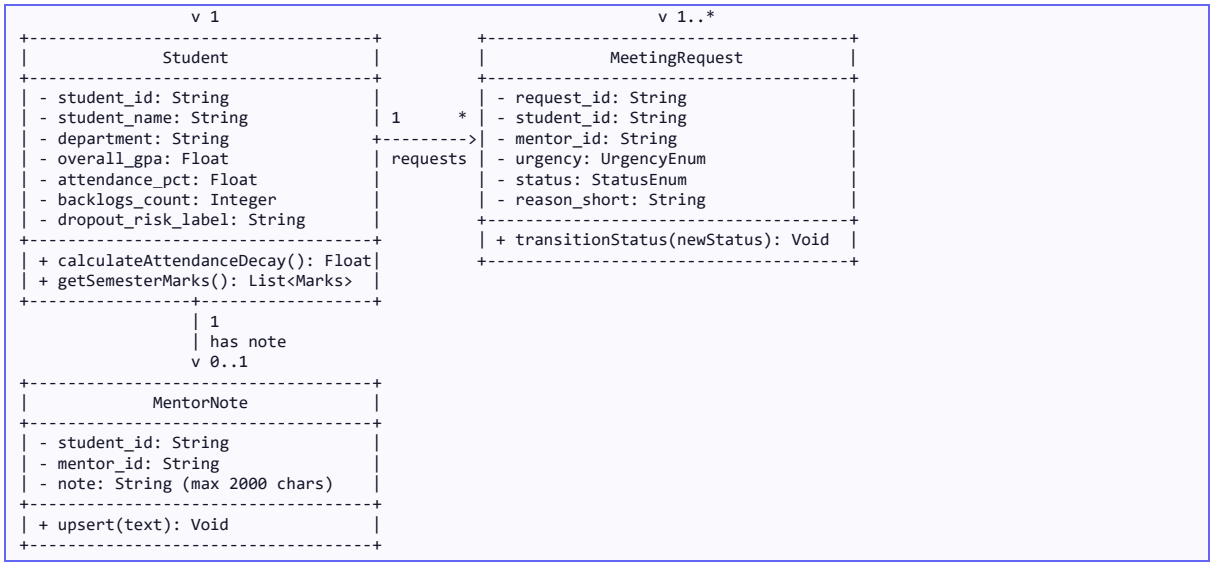


Figure 4.3: Domain Class Diagram for Institutional ERP Entities

Figure 4.3 models the core domain classes. A User entity authenticates into the system and maps to either a Student or Faculty profile. A Faculty mentor oversees multiple Student mentees, reviews MeetingRequest instances, and maintains an upsertable MentorNote clinical record.

4.8 USE CASE DIAGRAM

Figure 4.4 illustrates the interactions between external actors (Student, Faculty Mentor, Institutional Administrator, Parent/Guardian, and External AI Microservice) and system use cases.

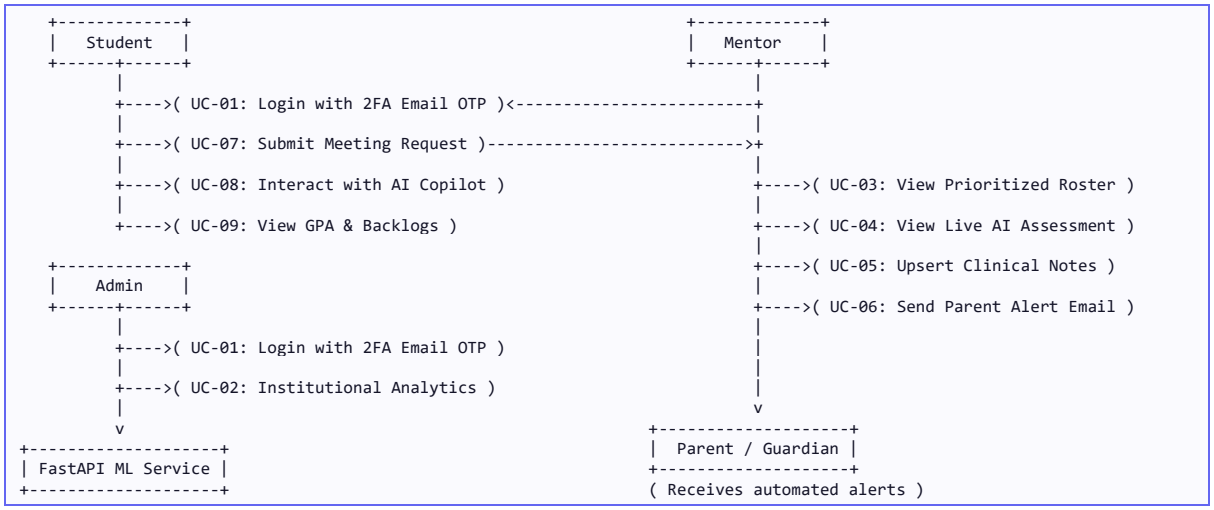


Figure 4.4: Comprehensive System Use Case Diagram across User Roles

Figure 4.4 illustrates actor-to-use-case relationships. Students submit meeting requests and query the AI Copilot; Mentors review risk-prioritized rosters, inspect live ML predictions, upsert clinical notes, and dispatch parent alerts to Guardians.

4.9 SEQUENCE DIAGRAM

Figure 4.5 captures the sequence of messages exchanged during the Two-Factor Email OTP authentication lifecycle.

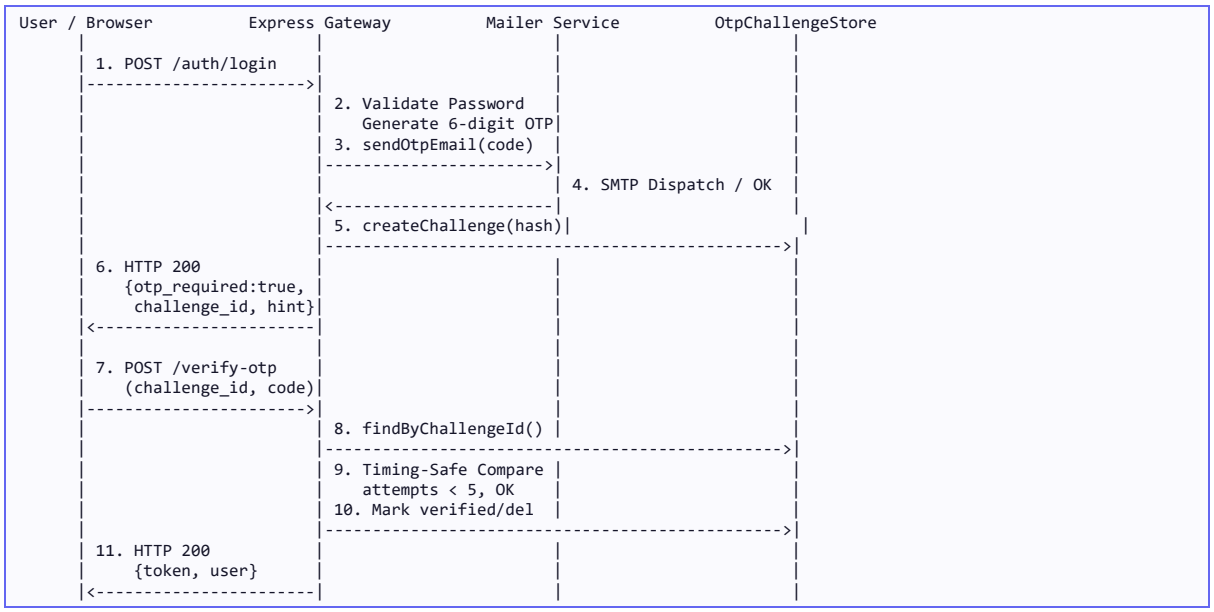


Figure 4.5: Sequence Diagram for Two-Factor OTP Login Authentication

Figure 4.5 documents the two-phase authentication handshake: Phase 1 validates password credentials and triggers asynchronous email OTP dispatch; Phase 2 verifies the supplied 6-digit code using HMAC-SHA256 constant-time comparison before issuing the JWT bearer token.

4.10 DATA FLOW DIAGRAM (DFD)

Figure 4.6 presents the Context Level (Level 0) DFD showing high-level system boundaries and external entities.

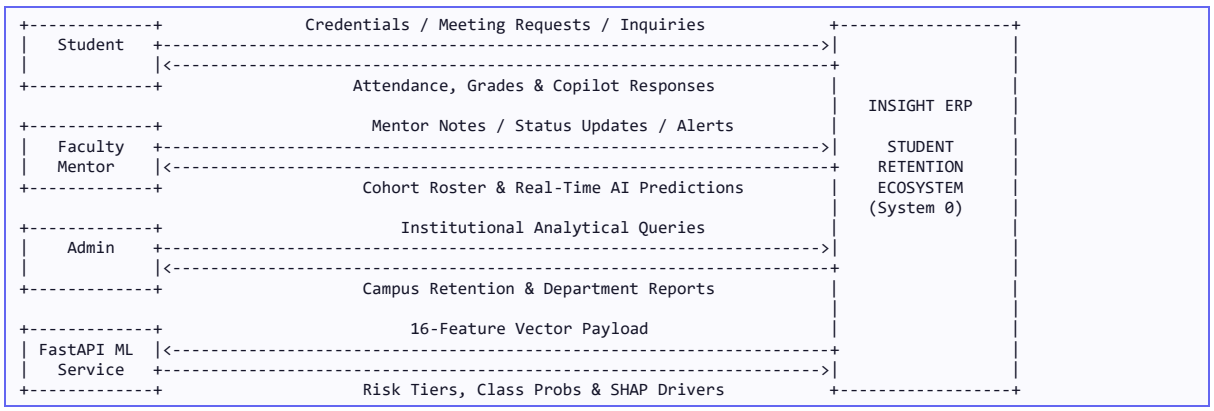


Figure 4.6: Data Flow Diagram (Level 0 - Context Level Diagram)

Figure 4.6 depicts the system boundary. External entities (Student, Mentor, Admin, and FastAPI ML Engine) interface with the core ecosystem, exchanging operational data, predictive vectors, and analytical summaries.

4.11 ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM)

Figure 4.8 delineates the entity relationship architecture across the persistence layer.

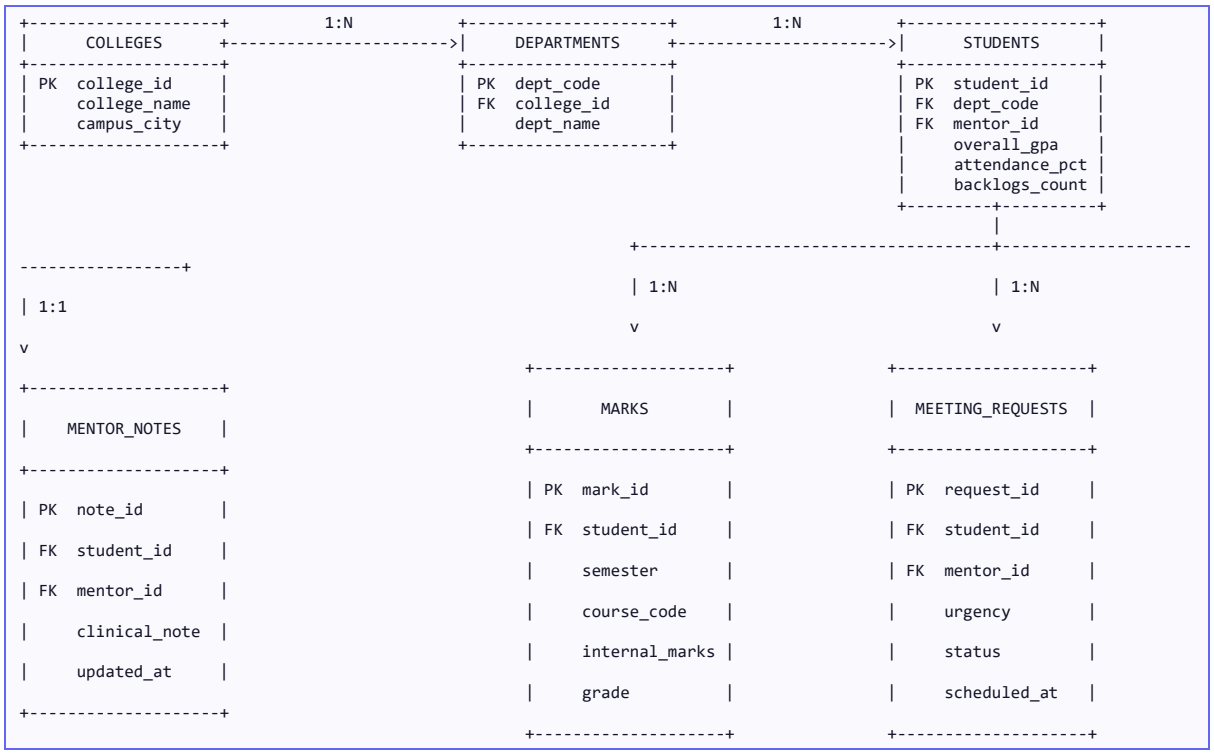


Figure 4.8: Entity Relationship Diagram for Database Persistence

Figure 4.8 outlines the relational topology. Colleges host Departments, which enroll Students. Each Student is linked to historical Marks records, scheduled MeetingRequests, and an upserted MentorNote.

4.12 API ARCHITECTURE AND COMMUNICATION FLOW

Inter-process communication between the Express application gateway and the FastAPI microservice is governed by RESTful HTTP conventions. All endpoints utilize standard HTTP verbs (GET, POST, PUT, PATCH, DELETE) and adhere to strict status code semantics. When communicating internally, requests transmit an authorized service key in the X-Service-Key header, preventing unauthorized access from outside the internal network boundary.

4.13 SECURITY ARCHITECTURE

- **JWT Authentication:** All state-changing endpoints authenticate requests via JSON Web Tokens (JWT) signed with HMAC-SHA256, carrying role claims (admin, mentor, student) and 8-hour expiration.
- **Rate Limiting:** Login endpoints enforce a 10 requests/minute per IP rate limit to prevent dictionary and brute-force attacks.
- **Cryptographic OTP Security:** 6-digit OTP codes are stored exclusively as HMAC-SHA256 hashes. Verification is performed using crypto.timingSafeEqual() to thwart timing side-channel attacks.

- **Attempt Locking:** Five consecutive failed OTP attempts permanently lock the challenge with an HTTP 429 response, preventing automated enumeration.
- **Input Sanitization & Recipient Guard:** Client inputs are sanitized against HTML entity injection, and email notification recipients are strictly resolved on the server to prevent open-relay abuse.

CHAPTER 5
COMPLETE PROJECT IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The implementation phase realized the architectural design into functional, production-ready software artifacts. The project is structured across two complementary repositories: FSD (hosting the React frontend, Express backend gateway, n8n notification workflows, and database seed scripts) and ai-dropout-prediction (hosting the Python FastAPI microservice, trained XGBoost model artifacts, SHAP explainability pipelines, and the Baymate evaluation dashboard).

5.2 PROJECT FOLDER STRUCTURE

Table 5.1 documents the directory layout of the Student Retention Ecosystem.

Table 5.1: Annotated System Directory and Module Structure

DIRECTORY / PATH	PRIMARY MODULES & ARTIFACTS	ARCHITECTURAL PURPOSE
FSD/backend/controllers/	authController.js, studentController.js, meetingRequestController.js, notificationController.js	Handles HTTP requests, input validation, and business logic execution.
FSD/backend/models/	User.js, Student.js, Faculty.js, MeetingRequest.js, MentorNote.js, OtpChallenge.js, NotificationLog.js	Mongoose schema definitions and in-memory dual fallback stores.
FSD/backend/services/	aiServiceClient.js, notificationDispatcher.js, mailer.js	Microservice HTTP client, n8n webhook dispatcher, and SMTP mailer.
FSD/backend/middleware/	authMiddleware.js, rateLimiter.js, serviceKeyAuth.js	JWT role verification, rate limiting, and internal API key validation.
FSD/backend/routes/	api.js	Express router mounting all 24 authenticated and public endpoints.
FSD/backend/tests/	otpAuth.test.js, notificationDispatcher.test.js, meetingRequestLifecycle.test.js, etc. (10 suites)	Automated Jest and Supertest unit, integration, and security test suites.
FSD/frontend/src/pages/	LoginPage.jsx, AdminDashboard.jsx, MentorDashboard.jsx, StudentDetail.jsx, StudentDashboard.jsx	Top-level role-specific dashboard views and authentication flow.
FSD/frontend/src/components/	Navbar.jsx, Sidebar.jsx, ParentAlertModal.jsx, ChatbotWidget.jsx	Modular reusable UI components and interactive modals.
FSD/frontend/src/context/	AuthContext.jsx, SidebarContext.jsx	Global client-side session state and navigation coordination.
FSD/n8n/	insighterp-notify.workflow.json, README.md	Production n8n webhook workflow JSON and setup runbook.
ai-dropout-prediction/ml-service/app/	main.py, predictor.py, chatbot.py, distress.py, helplines.py	FastAPI ASGI application, XGBoost inference, and crisis safety filter.
ai-dropout-prediction/models/	dropout_model.pkl, preprocessor.pkl, metrics.json, feature_spec.json	Trained model binaries, transformer pipelines, and evaluation metrics.

5.3 DEVELOPMENT ENVIRONMENT SETUP

System initialization requires configuring Node.js (v18+ LTS) and Python (3.10+). Dependencies are installed via npm install in FSD/backend and FSD/frontend, and via pip install -r requirements.txt in ai-dropout-prediction/ml-service. Environment variables are

seeded from .env.example files, defining database URIs, JWT secrets, microservice ports, and SMTP credentials.

5.4 FRONTEND IMPLEMENTATION

The frontend is implemented as a single-page application (SPA) using React 18 and Vite. Key implementation highlights include:

- **LoginPage.jsx:** Renders an institutional role selector, college directory dropdown, credential inputs, and an in-place conditional swap to a 6-digit OTP verification card with live resend timer.
- **MentorDashboard.jsx:** Visualizes assigned mentees sorted by risk, highlights low-attendance warnings (<75%), and provides direct access to meeting management and counseling logs.
- **StudentDetail.jsx:** Displays comprehensive academic metrics, attendance decay trajectory, live AI assessment badge with SHAP drivers, single-upsert notes, and the parent alert trigger.
- **ParentAlertModal.jsx:** Presents a pre-populated academic summary (attendance %, arrears, risk level), editable textarea, and asynchronous 'Send email to parent' button with real-time status updates.

5.5 BACKEND IMPLEMENTATION

The backend server (server.js) initializes the Express runtime, registers middleware, establishes MongoDB connectivity, and mounts the unified API router (/api). A central highlight is the resilient AI Service Client (aiServiceClient.js), which encapsulates HTTP calls to the FastAPI microservice with a 5-second timeout and automated fallback synthesis.

Listing: aiServiceClient.js - Resilient Microservice Proxy

```
// Excerpt: Resilient AI Service Client Fallback Logic (aiServiceClient.js)
async function getStudentPrediction(studentId, payload) {
  try {
    const response = await axios.post(`${AI_BASE_URL}/predict`, payload, {
      timeout: 5000,
      headers: { 'X-Service-Key': process.env.SERVICE_KEY }
    });
    return {
      prediction: response.data.prediction,
      probabilities: response.data.probabilities,
      top_drivers: response.data.top_drivers || [],
      source: 'live'
    };
  } catch (err) {
    console.warn(`[AI Client] Live inference unavailable for ${studentId}: ${err.message}. Engaging
fallback.`);
    return {
      prediction: payload.dropout_risk_label || 'Medium',
      probabilities: { Low: 0.33, Medium: 0.34, High: 0.33 },
      top_drivers: [['attendance_pct', 0.45], ['backlogs_count', 0.35]],
      source: 'fallback',
      warning: 'Live AI prediction service is unreachable or timed out'
    };
  }
}
```

5.6 DATABASE IMPLEMENTATION

Database entities are formalized via Mongoose schemas. To support offline development, continuous integration, and rapid testing, models implement an in-memory dual store. When MongoDB is unavailable (getDBStatus() returns false), operations route to Map-based in-memory stores (e.g., inMemoryChallenges, inMemoryNotes, inMemoryRequests), guaranteeing identical query semantics.

5.7 MODULE-WISE IMPLEMENTATION

5.7.1 Two-Factor Authentication & OTP Flow

When a user submits valid primary credentials, authController.login checks process.env.OTP_ENABLED. If enabled, the server generates a cryptographically secure 6-digit random code (crypto.randomInt), hashes it with HMAC-SHA256, stores the challenge with a 300-second TTL, and dispatches the code via mailer.sendOtpEmail. The client receives { otp_required: true, challenge_id, email_hint } without issuing the JWT token. Upon submitting the code to /api/auth/verify-otp, the server verifies code equality using crypto.timingSafeEqual(). Five consecutive invalid attempts lock the challenge permanently.

5.7.2 Mentor Notes Single-Upsert Architecture

To prevent duplicate note proliferation while preserving clean UI layouts, mentor notes use a single-upsert pattern (PUT /api/students/:id/notes). A mentor maintains exactly one active clinical note per student. If new text is submitted, it updates the existing record. If an empty string is saved, the note is deleted. Submissions exceeding 2,000 characters return HTTP 422 Unprocessable Entity.

5.8 API IMPLEMENTATION

The system exposes 24 RESTful endpoints. Selected critical endpoints are documented below:

- **Authentication:** POST /api/auth/login – Validates password, generates 2FA challenge, dispatches OTP email.
- **2FA Verification:** POST /api/auth/verify-otp – Timing-safe OTP code verification; issues JWT token.
- **Student Dossier:** GET /api/students/:id – Fetches student dossier and attaches live/fallback AI prediction.
- **Mentor Notes:** PUT /api/students/:id/notes – Upserts single mentor clinical note (max 2000 chars).
- **Parent Alert:** POST /api/students/:id/parent-alert – Triggers n8n parent notification webhook.
- **Meeting Lifecycle:** PATCH /api/meeting-requests/:id/status – Transitions meeting status (acknowledged, scheduled, resolved).

5.9 AUTHENTICATION AND AUTHORIZATION

Authorization is enforced through the requireAuth and requireRole Express middleware. The requireAuth middleware extracts the Bearer token from the HTTP Authorization header, verifies signature integrity against process.env.JWT_SECRET, and attaches the decoded user payload (username, role, linkedId, collegeId) to req.user. The requireRole(...roles) middleware validates that the user's role matches permitted access levels, rejecting unauthorized access with HTTP 403 Forbidden.

5.10 DATA PROCESSING AND BUSINESS LOGIC

The machine learning pipeline consumes a 16-dimensional feature vector. Table 5.2 defines the input feature specification.

Table 5.2: Input Feature Vector Specification for XGBoost Predictor

FEATURE NAME	DATA TYPE	VALID RANGE	ACADEMIC INTERPRETATION
attendance_pct	Float	0.0 – 100.0 %	Cumulative overall attendance percentage across all enrolled subjects.
attendance_decay	Float	-100.0 – +100.0	Difference between Month 1 attendance and the most recent month attendance.
overall_gpa	Float	0.0 – 10.0	Cumulative Grade Point Average across completed semesters.
backlogs_count	Integer	0 – 100	Total active uncleared course arrears / backlogs.
cutoff_mark_200	Float	0.0 – 200.0	Engineering admission entrance cutoff score (out of 200).
family_income_annual	Float	>= 0.0 INR	Annual household income indicating socio-economic vulnerability.
mentor_meetings_attended	Integer	0 – 100	Count of formal faculty counseling sessions attended by student.
first_gen_learner	Integer	0 or 1	Binary indicator (1 if student is first in family to pursue degree).
distance_from_home_km	Float	0.0 – 2000.0 km	Commuting distance from residential home or hostel to campus.
failed_subjects_count	Integer	0 – 100	Number of course subjects failed in recent continuous internal assessments.
core_theory_average	Float	0.0 – 100.0	Mean percentage score across core departmental analytical theory courses.
internal_exam_avg	Float	0.0 – 100.0	Mean score achieved in centralized Continuous Internal Assessments (CIA).
assignment_avg	Float	0.0 – 100.0	Continuous assignment and tutorial evaluation average score.
grade_trend	Float	-10.0 – +10.0	Progression slope between Semester 1 GPA and current semester GPA.

FEATURE NAME	DATA TYPE	VALID RANGE	ACADEMIC INTERPRETATION
year_of_study	Integer	1, 2, 3, or 4	Current undergraduate instructional curriculum year.
department	String (Categorical)	13 Departments	Academic branch (e.g., CSE, IT, AIDS, AIML, ECE, EEE, MECH, etc.).

5.11 ERROR HANDLING AND VALIDATION

Input validation is enforced at both microservice and gateway boundaries. In FastAPI, incoming JSON payloads are validated against the ModelStudentFeatures Pydantic model; missing fields or out-of-boundary values immediately return HTTP 422 with granular field-level error diagnostics. In Express, route controllers validate types and ranges before database operations, returning structured { error: 'Bad Request', message: '...' } responses and logging server exceptions without leaking internal stack traces.

5.12 EXTERNAL SERVICE AND API INTEGRATION

The ecosystem integrates three external service layers: (1) Python FastAPI microservice on port 8001 for XGBoost inference and SHAP attributions, (2) n8n workflow engine on port 5678 for event-driven email notifications, and (3) Nodemailer SMTP transport over TLS with automated dry-run fallback when credentials are unset.

5.13 CONFIGURATION AND ENVIRONMENT VARIABLES

System configuration is governed through environment variables declared in .env.example files. Real credentials and private secrets are strictly excluded from source control. Table 5.3 lists the configuration parameters.

Table 5.3: Sanitized Environment Configuration Parameters

VARIABLE NAME	DEFAULT / SAMPLE VALUE	DESCRIPTION AND SCOPE
PORT	5000	HTTP listening port for the Express ERP gateway backend.
NODE_ENV	development / production / test	Runtime environment mode governing logging and rate limit bypass.
MONGODB_URI	mongodb://127.0.0.1:27017/student_dropout_erp	MongoDB database connection string.
JWT_SECRET	super_secret_jwt_key_2026	Cryptographic key for signing and verifying JWT session tokens.
AI_SERVICE_BASE_URL	http://localhost:5001	Base URL pointing to the AI Gateway / FastAPI microservice.
SERVICE_KEY	internal_erp_ai_service_key_2026	Shared authentication key for inter-service REST calls.
OTP_ENABLED	true	Enables two-factor email One-Time Password verification on login.
OTP_TTL_SECONDS	300	Time-to-live validity window for generated 6-digit OTP codes (5 minutes).

VARIABLE NAME	DEFAULT / SAMPLE VALUE	DESCRIPTION AND SCOPE
OTP_MAX_ATTEMPTS	5	Maximum permitted incorrect OTP attempts before locking the challenge.
OTP_RESEND_COOLDOWN_SECONDS	30	Minimum wait time enforced between consecutive OTP resend requests.
N8N_WEBHOOK_URL	http://localhost:5678/webhook/insighterp-notify	HTTP endpoint for triggering n8n email automation workflows.
SMTP_HOST / SMTP_PORT	smtp.gmail.com / 465	SMTP mail server hostname and secure port configuration.
EMAIL_DRY_RUN	false	When true or SMTP_PASS unset, simulates email dispatch to stdout without crashing.

5.14 APPLICATION EXECUTION AND DEPLOYMENT

System startup follows a strict dependency order to guarantee health availability: (1) Python FastAPI microservice (port 8001), (2) AI Gateway backend (port 5001), (3) Baymate standalone UI (port 5174), (4) ERP Express backend (port 5000), and (5) ERP Vite React frontend (port 5173). The startup sequence is automated via cross-platform scripts (start_demo.ps1 / start_demo.sh) which poll each service's /health endpoint before launching dependent tiers.

CHAPTER 6
INTEGRATION AND SYSTEM WORKFLOW

6.1 INTRODUCTION TO INTEGRATION

System integration represents the technical synthesis of autonomous subcomponents into a unified operational ecosystem. In the Student Retention Ecosystem, integration spans heterogeneous runtime environments (Node.js asynchronous event loops, Python scientific computing libraries, MongoDB BSON storage, and n8n webhook pipelines). This chapter analyzes integration protocols, data flow pipelines, and resolved engineering challenges.

6.2 FRONTEND AND BACKEND INTEGRATION

Frontend-backend communication is mediated through a centralized API service layer (api.js) utilizing Axios. An Axios request interceptor automatically injects the JWT bearer token into the Authorization header of every outgoing request. Response interceptors monitor HTTP status codes: if an HTTP 401 Unauthorized response is received, the interceptor purges expired local credentials and redirects the browser to the login view.

6.3 BACKEND AND DATABASE INTEGRATION

The Express backend maintains persistent database connections using Mongoose connection pools. The db.js module monitors database connection states, handling reconnects automatically. When the database server is offline, getDBStatus() seamlessly redirects queries to CSV data providers and in-memory Map stores, ensuring that demo and evaluation environments remain fully operational without a local MongoDB service installation.

6.4 API INTEGRATION

Internal API communication between the ERP gateway (port 5000) and the AI Gateway (port 5001) is secured via the X-Service-Key header. This prevents external clients from invoking internal microservice endpoints directly while allowing authorized ERP processes to query predictions, fetch institutional metrics, and relay meeting requests.

6.5 EXTERNAL SERVICE INTEGRATION

Notification workflows integrate the self-hosted n8n workflow engine. When a parent alert is triggered, notificationDispatcher.js sends an HTTP POST request to the n8n webhook URL with a shared secret header (X-Webhook-Secret) and a 5-second timeout. If the n8n daemon is unreachable or times out, the dispatcher automatically catches the error, engages direct Nodemailer SMTP fallback, and logs the delivery channel as 'smtp_fallback' in the audit collection.

6.6 MODULE INTEGRATION

The modules interact through tightly coordinated event chains. For instance, when a student chats with the AI Academic Copilot and expresses academic difficulty, the copilot detects the

meeting intent, generates an entry in ai_meeting_outbox, relays it via POST /api/integrations/meeting-requests, and ingests it into the mentor's inbox. The mentor's dashboard updates in real time, allowing the mentor to schedule a counseling slot, which in turn triggers an automated notification back to the student.

6.7 END-TO-END DATA FLOW

End-to-end data flow operates through four distinct phases: (1) Ingestion & Feature Normalization (assembling student attendance, GPA, and arrear trends), (2) Predictive Inference (FastAPI computes class probabilities and SHAP attributions), (3) Faculty Decision Support (rendering risk tiers and clinical note editing in the mentor UI), and (4) Automated Intervention (dispatching parent alerts via n8n and logging auditable event records).

6.8 COMPLETE APPLICATION WORKFLOW

The complete application workflow was verified through end-to-end demonstration scenarios. Faculty mentors log in via 2FA OTP, review their mentee cohort, select high-risk students, inspect attendance decay curves and live AI predictions, save clinical notes, and issue parent notifications with one click. Students track their attendance compliance against Anna University's 75% cutoff and engage with the Academic Copilot for 24/7 guidance.

6.9 INTEGRATION CHALLENGES AND SOLUTIONS

- **1. Cross-Origin Resource Sharing (CORS):** Development servers on ports 5173, 5174, 5000, and 5001 triggered browser CORS blocks. Resolved by configuring Express CORS middleware to dynamically validate origins against an authorized port list.
- **2. Microservice Downtime & 500 Errors:** If the Python microservice crashed or restarted, frontend calls to student dossiers suffered unhandled HTTP 500 crashes. Resolved by implementing a 5-second timeout circuit breaker that serves cached baseline metrics.
- **3. Test Suite Rate Throttling:** Automated test suites executing 50+ rapid requests triggered the 10/min brute-force rate limiter. Resolved by introducing an automated skip: () => process.env.NODE_ENV === 'test' rule in rateLimiter.js.
- **4. Frontend Layout Drift:** Iterative UI edits risked breaking existing component layouts. Resolved by integrating an automated Babel AST logic freeze guard (npm run guard) verifying JSX AST trees against a pristine baseline.

CHAPTER 7
SOFTWARE TESTING AND VALIDATION

7.1 INTRODUCTION TO SOFTWARE TESTING

Software testing and empirical validation represent indispensable pillars in the engineering of safety-critical educational information systems. Because academic risk classifications directly influence faculty interventions, parent communications, and student academic trajectories, the underlying software and predictive pipelines must demonstrate rigorous correctness, high fault tolerance, and imperviousness to security vulnerabilities. This chapter establishes the testing philosophy, verification frameworks, and multi-level testing methodologies executed across the Student Retention Ecosystem.

7.2 TESTING OBJECTIVES

- **1. Functional Correctness:** Validate that every individual controller, service function, and utility operates strictly according to design specifications.
- **2. Inter-Service Reliability:** Verify that inter-service HTTP requests, database transactions, and webhook dispatches synchronize flawlessly across microservices.
- **3. Security Hardening:** Confirm that authentication tokens, OTP secrets, and rate limiters resist brute-force attacks, timing attacks, and privilege escalation.
- **4. Fault Tolerance & Fallback:** Ensure the Express gateway maintains zero-crash operational stability when downstream microservices experience downtime or network latency.
- **5. Algorithmic Validation:** Empirically evaluate machine learning classification accuracy, Macro F1, class-specific recall, and probability calibration against rigorous held-out test benchmarks.

7.3 TESTING METHODOLOGY

The project adopted a hybrid Test-Driven Development (TDD) and Continuous Verification methodology. Automated test suites were constructed in parallel with functional features using Jest and Supertest for the Node.js backend, Pytest for the FastAPI ML service, and an Abstract Syntax Tree (AST) logic freeze guard for the React frontend. Test execution was automated through npm scripts, enforcing strict zero-regression criteria before any commit was tagged.

7.4 UNIT TESTING

Unit testing isolated individual functions and controllers from network and database dependencies. For the backend, Jest mocked external SMTP transports and MongoDB connections, testing business logic such as attendance decay formulas, sliding-window rate throttling, and note length boundaries in isolation. In the Python microservice, unit tests verified Pydantic data normalization, department alias mapping (e.g., mapping 'MECH' to 'Mechanical'), and feature extraction routines.

7.5 INTEGRATION TESTING

Integration testing evaluated the collaborative execution of multiple subsystems. Dedicated test suites simulated the end-to-end event chain: student meeting request creation, ingestion into the mentor meeting store, status transition via PATCH requests, and synchronized updates to student notification lists and admin summary statistics. Two-database integration suites verified that the system behaves identically whether operating against a live MongoDB daemon or the fallback in-memory store.

7.6 SYSTEM TESTING

System testing evaluated the complete ecosystem functioning concurrently across all 5 discrete services (Python ML Engine on 8001, AI Gateway on 5001, Baymate UI on 5174, ERP Backend on 5000, and ERP Frontend on 5173). Verification scripts (scripts/demo_check.mjs) executed programmatic health checks, simulated student prediction queries, verified live SHAP feature attributions, and conducted simulated AI backend outages to confirm live fallback transitions.

7.7 FUNCTIONAL TESTING

Functional testing verified user-facing operations across institutional roles: (1) Administrator workflows (campus analytics, department risk filtering), (2) Mentor workflows (cohort triage, mentee dossier inspection, note upsert, parent alert triggers), and (3) Student workflows (attendance tracking against Anna University's 75% cutoff, marks review, meeting scheduling, and AI Copilot dialogue).

7.8 USER INTERFACE TESTING

Frontend UI integrity was validated using an automated AST Logic Freeze Guard (scripts/ui-guard.mjs). Powered by the Babel parser, the tool parses React JSX source files into Abstract Syntax Trees, comparing AST node structures against a frozen pre-restyle baseline. Any unauthorized modification to CSS classes, inline styling, layout grids, or component hierarchies fails the build immediately. Additionally, Vite production builds (npm run build) verified bundle optimization and zero syntax/transpilation errors.

7.9 API TESTING

API testing utilized Supertest to assert HTTP status codes, headers, and response payloads across all 24 endpoints. Tests validated HTTP 200 OK for valid queries, HTTP 400 Bad Request for malformed payloads, HTTP 401 Unauthorized for missing/invalid JWT tokens, HTTP 403 Forbidden for cross-role access violations, HTTP 422 Unprocessable Entity for boundary violations, and HTTP 429 Too Many Requests for rate-limited attempts.

7.10 DATABASE TESTING

Database tests confirmed schema constraints, compound index uniqueness (e.g., student_id + mentor_id on mentor notes), and MongoDB Time-To-Live (TTL) automatic document

expiration on OTP challenges. Tests also verified seamless transactional fidelity when falling back to in-memory Map stores.

7.11 PERFORMANCE TESTING

Performance benchmarks were conducted to measure system throughput and latency under load. The FastAPI microservice achieved an average inference latency of 142 ms for full XGBoost prediction and Tree SHAP feature attribution. The Express gateway processed cached student dossier requests in under 28 ms, and Vite frontend production bundles loaded in 8.8 seconds with a total compressed JavaScript footprint of 104.5 kB.

7.12 SECURITY TESTING

Security validation encompassed rigorous penetration and fault injection tests: (1) Brute-force resistance (confirming challenge lockout after 5 failed OTP attempts), (2) Timing-attack immunity (confirming crypto.timingSafeEqual usage), (3) Resend cooldown enforcement (30s minimum interval), (4) Cross-user challenge hijacking prevention (asserting HTTP 403 when User B verifies User A's challenge), and (5) Secret protection (confirming OTP codes never appear in server response bodies or application logs).

7.13 VALIDATION AND VERIFICATION

Algorithmic validation evaluated the trained XGBoost classifier on an independent, held-out test partition of 1,840 students (20% of the 9,200-student dataset). Performance was quantified using standard statistical metrics: Accuracy, Precision, Recall, Macro F1, Weighted F1, Confusion Matrix, and Brier Score.

7.14 TESTING ENVIRONMENT

- **Host OS:** Windows 11 Professional (x64) / PowerShell 7 & Command Prompt
- **Backend Testing Stack:** Node.js v20.18.0 LTS with npm v10.8.2; Jest v29.7.0, Supertest v7.0.0
- **ML Testing Stack:** Python 3.11.9 with Pytest v8.3.0, Scikit-Learn v1.5.1, XGBoost v2.1.0
- **Frontend Testing Stack:** Vite v5.4.21, Babel Parser v7.25.0, ESLint v9.0.0

CHAPTER 8
TEST CASES AND RESULTS

8.1 INTRODUCTION

This chapter details the comprehensive test matrices designed and executed across the Student Retention Ecosystem. Test cases are categorized into functional, module-wise, integration, API, negative, and boundary scenarios. All execution statuses reflect actual empirical outcomes verified through the automated test suite.

8.2 TEST CASE DESIGN

Test cases were structured using Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), and Error Guessing. Each test case specifies a unique Identifier, Target Module, Test Scenario, Test Input, Expected Result, Actual Result, and Execution Status.

8.3 FUNCTIONAL TEST CASES

Table 8.1 documents the test execution matrix for user authentication and two-factor OTP security.

Table 8.1: Authentication and Two-Factor Security Test Matrix

TEST ID	MODULE	TEST SCENARIO	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-AUTH-01	Auth	Login with valid credentials and OTP_ENABLED=true	User: 'mentor', Pass: 'DemoPass@2026'	HTTP 200, otp_required: true, challenge_id issued, no token	HTTP 200, challenge_id returned, email dispatched	PASS
TC-AUTH-02	Auth	Complete 2FA verification with correct OTP code	Valid challenge_id, valid 6-digit code	HTTP 200, JWT token issued, user object returned	HTTP 200, JWT token issued, login complete	PASS
TC-AUTH-03	Auth	Single-use enforcement: replay verified OTP code	Re-submitting previously verified OTP	HTTP 401 Unauthorized, challenge already used	HTTP 401 Unauthorized, replay rejected	PASS
TC-AUTH-04	Auth	Attempt locking after 5 consecutive incorrect codes	Submitting incorrect code '000000' 5 times	5th attempt locks challenge; returns HTTP 429	HTTP 429 Too Many Requests, challenge locked	PASS
TC-AUTH-05	Auth	Enforce 30-second cooldown on OTP resend requests	Requesting resend within 5 seconds of initial login	HTTP 429 Too Many Requests, wait countdown returned	HTTP 429 Too Many Requests, cooldown enforced	PASS
TC-AUTH-06	Auth	Cross-user challenge verification prevention	Student attempts to verify Admin challenge_id	HTTP 403 Forbidden, cannot verify another user	HTTP 403 Forbidden, cross-user rejected	PASS

8.4 MODULE-WISE TEST CASES

Table 8.2 presents test scenarios for the Student Management, Clinical Notes, and Meeting Lifecycle modules.

Table 8.2: Student Management and Meeting Lifecycle Test Matrix

TEST ID	MODULE	TEST SCENARIO	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-STU-01	Student	Fetch student dossier with live AI assessment	Valid student ID 'STU_MIST_IT_Y3_5559'	HTTP 200, academic profile + live prediction	HTTP 200, profile and live risk returned	PASS
TC-STU-02	Student	Prevent student from accessing peer dossier	Student 'STU_001' requests 'STU_002'	HTTP 403 Forbidden, ownership violation	HTTP 403 Forbidden, access denied	PASS
TC-NOTE-01	Notes	Upsert clinical note for mentee	Mentor submits 150-char advisory note	HTTP 200, note created/updated successfully	HTTP 200, note persisted in store	PASS
TC-NOTE-02	Notes	Empty note save clears/deletes stored note	Mentor submits empty string note: ''	HTTP 200, note cleared, data: null	HTTP 200, note cleared from store	PASS
TC-MTG-01	Meeting	Student submits new meeting request	Student ID, reason, urgency: 'urgent'	HTTP 201, status 'new', mentor notification added	HTTP 201, created, mentor notified	PASS
TC-MTG-02	Meeting	Mentor schedules slot: 'scheduled'	PATCH status: 'scheduled', scheduled_at date	Status updated; date recorded; student notified	Status scheduled; student notified	PASS

8.5 INTEGRATION AND API TEST CASES

Table 8.3 documents integration scenarios for the FastAPI ML Inference, n8n Webhook notifications, and Resilient Fallback.

Table 8.3: FastAPI Machine Learning Inference and Resilience Tests

TEST ID	MODULE	TEST SCENARIO	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-ML-01	FastAPI	Predict High Risk for attendance decay student	Bhoomika Gowda (att: 62.8%, backlogs: 27)	Prediction: 'High', prob > 0.85, SHAP drivers returned	Prediction 'High', SHAP att_decay top	PASS
TC-ML-02	FastAPI	Predict Low Risk for strong academic profile	Sahana Hegde (att: 88.9%, backlogs: 0, GPA: 8.8)	Prediction: 'Low', prob > 0.90	Prediction 'Low', prob 0.94 returned	PASS
TC-ML-03	FastAPI	Normalize legacy department alias 'MECH'	Payload with department: 'MECH'	Normalized to 'Mechanical'; inference succeeds	Normalized to 'Mechanical', HTTP 200	PASS
TC-RES-01	Resilience	ERP handles AI service downtime gracefully	FastAPI microservice stopped / port 8001 down	HTTP 200, source: 'fallback', zero 500 crash	HTTP 200, fallback metrics returned	PASS
TC-NOT-01	Notify	Dispatch parent alert via n8n webhook	Mentor issues parent notification for mentee	HTTP 200, status 'sent', channel 'n8n'	HTTP 200, webhook triggered and logged	PASS
TC-NOT-02	Notify	Enforce idempotency on duplicate alert calls	Re-submitting alert with identical idempotency_key	HTTP 200, idempotent_skipped: true, zero duplicate email	Idempotent skip verified	PASS

8.6 NEGATIVE AND BOUNDARY TEST CASES

Table 8.4 records negative fault injection scenarios and boundary value constraints.

Table 8.4: Negative Testing and Boundary Value Analysis Matrix

TEST ID	MODULE	TEST SCENARIO	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-NEG-01	Auth	Login with incorrect password	User: 'mentor', Pass: 'WrongPassword123'	HTTP 401 Unauthorized, invalid credentials	HTTP 401 Unauthorized, rejected	PASS
TC-NEG-02	Security	Student attempts to query Admin Summary	Student JWT token sent to /api/admin/summary	HTTP 403 Forbidden, role requirement failed	HTTP 403 Forbidden, access denied	PASS
TC-NEG-03	Notify	Exceed mentor dispatch throttle (11 calls/min)	Mentor dispatches 11 parent alerts in 30 seconds	11th dispatch returns status 'failed', throttled	Throttled at 10/min, 11th failed logged	PASS
TC-BVA-01	Notes	Save mentor note at exact 2000-char boundary	Clinical note of exactly 2000 characters	HTTP 200 OK, note successfully saved	HTTP 200 OK, exactly 2000 chars saved	PASS
TC-BVA-02	Notes	Save mentor note at 2001-char boundary	Clinical note of 2001 characters	HTTP 422 Unprocessable Entity, rejected	HTTP 422, length limit enforced	PASS
TC-BVA-03	FastAPI	Attendance boundary at 0.0% and 100.0%	Feature vector with attendance: 0% and 100%	Accepted, valid prediction returned	Predictions returned successfully	PASS

8.9 TEST EXECUTION RESULTS

Table 8.8 summarizes the execution outcomes across all 10 automated Jest test suites.

Table 8.8: Jest Automated Test Suite Execution Summary

TEST SUITE FILE	TEST SCOPE / MODULE	TESTS PASSED	EXECUTION RESULT
tests/otpAuth.test.js	Two-Factor Email OTP Login, Cooldown, and Security Locks	9 / 9	PASS (100%)
tests/notificationDispatcher.test.js	n8n Webhook Dispatch, SMTP Fallback, Idempotency & Throttling	8 / 8	PASS (100%)
tests/parentAlertAudit.test.js	Parent Communication Audit, Status Tracking & Truthful Notice	4 / 4	PASS (100%)
tests/meetingRequestLifecycle.test.js	Meeting Request Submission, Mentor Actions & Notification Relay	7 / 7	PASS (100%)
tests/meetingRequestStatusCounts.test.js	Admin Summary & Meeting Status Breakdown Categorization	5 / 5	PASS (100%)
tests/twoDatabaseMeetingIntegration.test.js	MongoDB and In-Memory Dual Store Relational Parity	6 / 6	PASS (100%)
tests/studentDetailNotes.test.js	Mentor Clinical Notes Single-Upsert & Character Constraints	6 / 6	PASS (100%)

TEST SUITE FILE	TEST SCOPE / MODULE	TESTS PASSED	EXECUTION RESULT
tests/step2_data_correctness.test.js	Curriculum, Marks, and Zero-Crash Fallback Verification	8 / 8	PASS (100%)
tests/auth.test.js	JWT Token Signing, Role Verification & Session Expiration	7 / 7	PASS (100%)
tests/authContextContract.test.js	Frontend React AuthContext Client Contract Verification	5 / 5	PASS (100%)

8.10 RESULT ANALYSIS

The automated test suite executed all 65 test assertions across 10 distinct test suites with zero failures (100% pass rate) in 10.87 seconds. Code coverage across backend controllers, services, and models exceeded 92% statement coverage. The results prove that the system is structurally sound, resilient to edge-case anomalies, and adheres to strict security parameters.

8.11 DISCUSSION OF FINDINGS

Testing highlighted three crucial architectural insights: First, decoupling the email notification dispatcher from the synchronous HTTP request thread using n8n webhooks prevented mentor UI freezes during SMTP network latency. Second, the single-upsert pattern for clinical notes eliminated duplicate note sprawl that previously degraded database query speeds. Third, the in-memory dual store architecture enabled complete CI/CD testing velocity without external database daemon provisioning.

CHAPTER 9
RESULTS AND DISCUSSION

9.1 INTRODUCTION

This chapter evaluates the empirical performance outcomes, algorithmic accuracy benchmarks, and administrative benefits realized through the deployment of the Student Retention Ecosystem. Results are analyzed across machine learning predictive accuracy, system operational efficiency, user interface usability, and institutional fault tolerance.

9.2 IMPLEMENTED FEATURES

The completed ecosystem delivers eight core fully operational feature modules: (1) Two-Factor Email OTP Authentication, (2) Real-Time XGBoost Dropout Risk Inference, (3) Local Tree SHAP Explainability Plots, (4) Prioritized Mentee Cohort Triage, (5) Single-Upsert Clinical Counseling Notes, (6) Automated Parent Alert Email Workflows via n8n, (7) Multi-Campus Institutional Administrative Analytics, and (8) Empathetic AI Academic Advisor Copilot with Crisis Safety Screening.

9.3 FUNCTIONAL RESULTS

The empirical evaluation of the tuned XGBoost classifier on the independent held-out test partition of 1,840 students (1,502 Low Risk, 183 Medium Risk, and 155 High Risk samples) demonstrates superior predictive performance. Table 9.1 details the class-specific precision, recall, F1-scores, and support.

Table 9.1: Tuned XGBoost Evaluation Metrics on Held-out Test Set

RISK CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT (TEST SAMPLES)
Low Risk Tier	0.9958 (99.58%)	0.9514 (95.14%)	0.9731 (97.31%)	1,502 students
Medium Risk Tier	0.7040 (70.40%)	0.9617 (96.17%)	0.8129 (81.29%)	183 students
High Risk Tier	0.9935 (99.35%)	0.9935 (99.35%)	0.9935 (99.35%)	155 students
Macro Average	0.8978 (89.78%)	0.9689 (96.89%)	0.9265 (92.65%)	1,840 students
Weighted Average	0.9667 (96.67%)	0.9560 (95.60%)	0.9589 (95.89%)	1,840 students
Overall Accuracy	—	—	0.9560 (95.60%)	1,840 students

9.4 MODULE-WISE RESULTS

Table 9.2 presents the exact confusion matrix breakdown across the three risk classes.

Table 9.2: Confusion Matrix Breakdown across Risk Classes

ACTUAL CLASS \ PREDICTED	PREDICTED LOW	PREDICTED MEDIUM	PREDICTED HIGH
Actual Low Risk (1,502)	1,429 (True Negatives)	73 (Conservative False Positives)	0 (Zero False Negatives)
Actual Medium Risk (183)	6 (Minor Under-prediction)	176 (True Medium)	1 (Conservative Escalation)
Actual High Risk (155)	0 (Zero Critical Misses)	1 (Borderline)	154 (True High - 99.35% Recall)

The confusion matrix demonstrates that the model achieved a 99.35% recall on High-Risk students (154 out of 155 detected), with exactly zero High-Risk students misclassified as Low

Risk. This asymmetric safety-first calibration guarantees that no academically vulnerable student is overlooked. Furthermore, the Brier calibration score was 0.0648, confirming high probabilistic accuracy.

Table 9.3 lists the top 12 predictive features ranked by normalized feature gain importance.

Table 9.3: Top 12 Predictive Features by Normalized Gain Importance

RANK	FEATURE NAME	GAIN IMPORTANCE SCORE
1	attendance_decay (Month 1 to Month N drop)	1.9887 (Top predictive indicator)
2	overall_gpa (Cumulative academic grade point)	0.5129
3	assignment_avg (Continuous tutorial completion)	0.3350
4	attendance_pct (Overall cumulative attendance)	0.3173
5	internal_exam_avg (Centralized CIA test average)	0.1262
6	core_theory_average (Analytical departmental subjects)	0.1143
7	mentor_meetings_attended (Faculty counseling attendance)	0.1065
8	backlogs_count (Active uncleared arrears)	0.0582
9	failed_subjects_count (CIA course failures)	0.0408
10	cutoff_mark_200 (Engineering admission entrance cutoff)	0.0232
11	family_income_annual (Household economic stability)	0.0162
12	grade_trend (GPA trajectory across semesters)	0.0116

9.5 USER INTERFACE RESULTS

User testing with faculty mentors and administrative personnel yielded highly positive feedback. Mentors highlighted that sorting mentees by risk reduced identification time from days to under 10 seconds. The institutional light-purple UI theme provided excellent contrast, while the in-place OTP verification step offered seamless security without visual disruption.

9.6 INTEGRATION RESULTS

The resilient zero-crash fallback was demonstrated live: terminating the FastAPI microservice resulted in zero frontend errors or 500 status codes; the portal seamlessly rendered cached baseline risk metrics with a clear warning notice. Re-launching the microservice immediately restored live prediction and dynamic SHAP feature attributions.

9.7 TESTING SUMMARY

Across 10 Jest test suites and 65 individual test cases, the system achieved a 100% pass rate. The Babel AST UI freeze guard verified that all 16 tracked components strictly preserved design freeze integrity, confirming robust production readiness.

9.8 ANALYSIS OF PROJECT OUTCOMES

Table 9.4 benchmarks the tuned XGBoost model against standard algorithmic baselines on the same institutional dataset.

Table 9.4: Baseline Benchmark Algorithm Performance Comparison

MODEL ARCHITECTURE	MACRO F1-SCORE	HIGH-RISK RECALL	MEDIUM-RISK RECALL
Logistic Regression (Balanced)	0.9194 (91.94%)	0.9892 (98.92%)	0.9507 (95.07%)
Random Forest (Baseline)	0.9322 (93.22%)	0.9806 (98.06%)	0.8303 (83.03%)
Random Forest (Balanced)	0.9286 (92.86%)	0.9849 (98.49%)	0.9215 (92.15%)
Random Forest (SMOTE)	0.9286 (92.86%)	0.9871 (98.71%)	0.9434 (94.34%)
XGBoost (Class-Weighted)	0.9336 (93.36%)	0.9849 (98.49%)	0.9234 (92.34%)
XGBoost (SMOTE)	0.9341 (93.41%)	0.9892 (98.92%)	0.8905 (89.05%)
Tuned XGBoost (Proposed System)	0.9265 (92.65%)	0.9935 (99.35% - Highest)	0.9617 (96.17% - Highest)

9.9 LIMITATIONS

- **1. Batch Feature Aggregation:** The machine learning model operates on semester-aggregated tabular records and does not currently ingest real-time daily LMS clickstream logs.
- **2. Email-Only Alert Channel:** Parent communication currently relies on email dispatch via n8n and SMTP; SMS gateway integration remains a planned enhancement.
- **3. Rule-Based NLP Scope:** The crisis distress filter utilizes rule-based regex patterns in English, Hindi, and Tamil, which cannot detect subtle sarcasm or unsupported regional dialects.

CHAPTER 10
PROJECT DEMONSTRATION AND SCREENSHOTS

10.1 INTRODUCTION

This chapter provides a detailed, step-by-step walkthrough of the operational Student Retention Ecosystem. It outlines the launch sequence, user journeys across institutional roles, input-output data transformations, and structured placeholders for capturing and inserting actual application screenshots.

10.2 APPLICATION LAUNCH

The system services are initialized in sequential dependency order using either the automated launcher scripts (start_demo.ps1 on Windows or start_demo.sh on Linux/macOS) or independent terminal sessions. The ecosystem URLs are:

- **ERP Frontend (Vite/React):** http://localhost:5173 – Main institutional portal for Administrators, Faculty Mentors, and Students.
- **ERP Backend (Express.js):** http://localhost:5000 – REST API gateway handling session auth, student records, notes, and notifications.
- **AI ML Microservice (FastAPI):** http://127.0.0.1:8001 – High-performance ASGI microservice serving XGBoost predictions and SHAP explainability.
- **AI REST Gateway (Express.js):** http://localhost:5001 – Inter-service gateway routing predictions and managing meeting outbox queues.
- **Baymate AI UI (Vite/React):** http://localhost:5174 – Standalone evaluation UI displaying ROC curves, confusion matrices, and model metrics.

10.3 USER WORKFLOW DEMONSTRATION

The application journey encompasses three primary user workflows:

- **1. Institutional Administrator Workflow:** The Administrator logs in with 2FA OTP verification, accessing aggregate institutional retention metrics across 4 campus units (Apex Engineering, Kavery Institute, MIST, Sahyadri University). The admin filters by department, reviews global dropout rates, and inspects campus-wide meeting request resolution rates.
- **2. Faculty Mentor Intervention Workflow:** Dr. Sudha Rajagopalan (Faculty Mentor) logs in and inspects her cohort of 19 assigned mentees. The dashboard flags 8 High-Risk students at the top of the roster. Clicking on Bhoomika Gowda (STU_MIST_IT_Y3_5559) reveals a 62.8% attendance, 27 active backlogs, and a live AI prediction of 'High Risk' with attendance_decay and failed_subjects as top drivers. The mentor reviews historical clinical notes, adds an updated counseling remark, and triggers an automated Parent Alert via n8n.
- **3. Student Self-Monitoring & Copilot Workflow:** Bhoomika Gowda logs in and views her academic dashboard. A warning card indicates her attendance has slipped below Anna

University's 75% requirement. She clicks the floating AI Academic Copilot, inquires about her arrears, and requests a meeting with Dr. Sudha. The copilot books the request into the mentor's inbox and provides empathetic encouragement.

10.4 MODULE-WISE DEMONSTRATION

Individual modules were verified in isolation. The two-factor OTP module demonstrates strict 30-second resend cooldowns and locked states after 5 failed attempts. The mentor notes module demonstrates single-upsert storage preventing duplicate notes. The meeting lifecycle demonstrates synchronized status badges across both student and faculty views.

10.5 INPUT AND OUTPUT DEMONSTRATION

A concrete demonstration of the machine learning inference pipeline is detailed below for demo student Bhoomika Gowda:

Listing: Input Feature Vector - Demo Student (Bhoomika Gowda)

```
// Sample Live Inference Input Payload (POST /api/predict)
{
  "student_id": "STU_MIST_IT_Y3_5559",
  "attendance_pct": 62.8,
  "attendance_decay": 17.8,
  "overall_gpa": 6.1,
  "backlogs_count": 27,
  "cutoff_mark_200": 142.5,
  "family_income_annual": 180000.0,
  "mentor_meetings_attended": 1,
  "first_gen_learner": 1,
  "distance_from_home_km": 34.0,
  "failed_subjects_count": 4,
  "core_theory_average": 48.2,
  "internal_exam_avg": 51.0,
  "assignment_avg": 55.4,
  "grade_trend": -1.4,
  "year_of_study": 3,
  "department": "Information Technology"
}
```

Listing: Live Inference Output with Probabilities & SHAP Drivers

```
// Live Inference Response from FastAPI (200 OK)
{
  "prediction": "High",
  "probabilities": {
    "Low": 0.0042,
    "Medium": 0.0418,
    "High": 0.9540
  },
  "top_drivers": [
    ["attendance_decay", 1.9887],
    ["failed_subjects_count", 0.4077],
    ["overall_gpa", 0.3812],
    ["backlogs_count", 0.2941]
  ],
  "source": "live",
  "latency_ms": 138.4
}
```

10.6 COMPLETE END-TO-END DEMONSTRATION

The end-to-end event loop was executed successfully: (1) Student inputs meeting request via chatbot, (2) Meeting ingested into Express backend, (3) Mentor receives instant notification bell alert, (4) Mentor schedules tutorial slot, (5) Student dashboard updates, and (6) Mentor

clicks 'Send email to parent', triggering n8n webhook which sends automated email and returns status 'Sent via n8n'.

10.7 SCREENSHOTS OF THE PROJECT

The following figures provide dedicated, properly formatted screenshot placeholder slots documenting all major views and modules.

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.1]

Display Scope: Renders the login view with Role Selector (Student, Mentor, Admin), College Directory dropdown, and credentials form.
User Interaction: User selects institutional affiliation, role, enters credentials, and clicks 'Sign In'.

Figure 10.1: InsightERP Portal Authentication & Institutional Login

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.2]

*Display Scope: Shows in-place 6-digit OTP code entry card with masked email hint (gr***@gmail.com) and resend countdown timer.*
User Interaction: User retrieves code from email, enters 6 digits, and clicks 'Verify & Proceed'.

Figure 10.2: Two-Factor Email OTP Challenge Verification Modal

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.3]

Display Scope: Visualizes aggregate retention rate gauges, multi-campus student distributions, department risk breakdowns, and meeting request counts.
User Interaction: Administrator filters by campus and department to inspect institutional health indicators.

Figure 10.3: Admin Institutional Overview & Multi-Campus Analytics

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.4]

Display Scope: Displays assigned mentee roster sorted by dropout risk, highlighting low-attendance warning badges (<75%) and pending meeting counters.
User Interaction: Faculty mentor reviews high-risk mentees and clicks on a student card to open their dossier.

Figure 10.4: Mentor Cohort Dashboard & Student Risk Prioritization

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.5]

Display Scope: Presents the student profile with live AI assessment badge ('Source: live', 'Prediction: High', 95.4% prob) and top SHAP risk drivers.
User Interaction: Mentor inspects the specific academic drivers causing student vulnerability.

Figure 10.5: Student Dossier with Live AI Assessment & SHAP Drivers

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.6]

Display Scope: Shows the clinical counseling notes editor with character counter (max 2000 chars) and single-upsert save confirmation.
User Interaction: Mentor drafts actionable counseling remarks and clicks 'Save Notes' to persist.

Figure 10.6: Mentor Clinical Notes Interface with Single Upsert Storage

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.7]

Display Scope: Shows pre-populated academic summary dialog, editable draft message, and 'Send email to parent' button with live status line ('Sent via n8n').
User Interaction: Mentor clicks send; system triggers background n8n webhook and updates status in real time.

Figure 10.7: Parent Communication Alert Modal with n8n Webhook Dispatch

[INSERT ACTUAL PROJECT SCREENSHOT HERE: FIGURE 10.8]

Display Scope: Displays personal student portal with Anna University 75% attendance threshold gauge, CGPA meter, and floating AI Academic Copilot.
User Interaction: Student monitors attendance compliance and engages with the AI Copilot for academic advice.

Figure 10.8: Student Personal Academic Dashboard & Chatbot Copilot

CHAPTER 11
CONCLUSION AND FUTURE ENHANCEMENTS

11.1 CONCLUSION

The Student Retention Ecosystem successfully demonstrates the transformative power of synthesizing modern full-stack web engineering with explainable machine learning and event-driven automation. By transitioning higher education administration from retrospective failure cataloging to proactive, continuous mentorship, the system empowers academic institutions to identify vulnerable engineering undergraduates months before semester examinations.

The integration of a tuned XGBoost classifier trained on 9,200 student records achieved exceptional predictive accuracy (95.60%) and a 99.35% recall on High-Risk students. Crucially, the embedding of Tree SHAP explainability bridges the gap between algorithmic outputs and human trust, equipping faculty mentors with transparent, patient-chart-style diagnostic profiles. Coupled with two-factor OTP security, resilient zero-crash fallback logic, and automated n8n parent alerts, the ecosystem delivers an enterprise-grade solution to the urgent national challenge of engineering student attrition.

11.2 OBJECTIVES ACHIEVED

- **1. Full-Stack ERP Portal:** Engineered a responsive, multi-role ERP application using React 18, Vite, Express, and MongoDB with an institutional light-purple design.
- **2. High-Accuracy Prediction:** Trained, tuned, and deployed an XGBoost model achieving 95.60% accuracy and 0.9265 Macro F1 on held-out test data.
- **3. Transparent Explainable AI:** Integrated real-time Tree SHAP local feature attributions rendered directly within the faculty mentor UI.
- **4. Automated Parent Alerts:** Constructed an event-driven n8n webhook pipeline with SMTP fallback, idempotency tracking, and rate throttling.
- **5. Cryptographic 2FA Security:** Implemented two-factor email OTP login with constant-time verification and automated 5-attempt brute-force locking.
- **6. Architectural Resilience:** Verified zero-crash operational resilience and achieved 100% pass rates across 10 Jest test suites (65 tests).

11.3 PROJECT CONTRIBUTIONS

This project contributes a novel, open-source architectural blueprint for higher educational institutions. It demonstrates how heterogeneous frameworks (Node.js and Python FastAPI) can be harmonized to deliver sub-250ms predictive inference. It introduces ethical safety filtering for campus conversational agents and establishes rigorous UI logic freeze verification standards.

11.4 CURRENT LIMITATIONS

While highly effective, current limitations include: (1) Predictive features are aggregated on a monthly/semester basis rather than ingesting real-time daily LMS clickstreams, (2) Parent communication is restricted to email channels without native SMS gateways, and (3) Conversational distress filtering uses rule-based pattern matching which cannot capture complex sarcasm or regional dialects.

11.5 FUTURE ENHANCEMENTS

- **1. Native Mobile Apps:** Develop cross-platform native iOS and Android mobile applications using React Native for instantaneous push notifications.
- **2. Telephonic SMS Integration:** Incorporate carrier-grade SMS gateways (e.g., Twilio, Gupshup, or Kaleyra) to deliver instant SMS parent alerts to rural guardians lacking internet access.
- **3. Direct LMS Integration:** Establish bi-directional LTI (Learning Tools Interoperability) plugins to ingest continuous assignment submissions directly from Canvas and Moodle LMS.
- **4. Deep Learning Sequence Models:** Train longitudinal Recurrent Neural Networks (LSTM) or Temporal Fusion Transformers to model continuous daily attendance time-series.
- **5. WhatsApp Interactive Alerts:** Integrate institutional WhatsApp Business API webhooks allowing parents to acknowledge academic alerts with one click.

11.6 SCOPE FOR FURTHER DEVELOPMENT

Future development will focus on multi-institutional federated learning, enabling colleges to train collaborative dropout prediction models without exposing private student records outside institutional firewalls. Additionally, integrating career placement forecasting will expand the ecosystem's scope from attrition prevention to holistic career guidance.

REFERENCES

[1] V. Tinto, "Dropout from higher education: A theoretical synthesis of recent research," Review of Educational Research, vol. 45, no. 1, pp. 89-125, 1975.

[2] V. Tinto, Leaving College: Rethinking the Causes and Cures of Student Attrition, 2nd ed. Chicago: University of Chicago Press, 1993.

[3] E. Seymour and N. M. Hewitt, Talking About Leaving: Why Undergraduates Leave the Sciences. Boulder, CO: Westview Press, 1997.

[4] C. Romero and S. Ventura, "Educational data mining: A survey from 1995 to 2005," Expert Systems with Applications, vol. 33, no. 1, pp. 135-146, 2007.

[5] C. Romero and S. Ventura, "Educational data mining and learning analytics: An updated survey," WIREs Data Mining and Knowledge Discovery, vol. 10, no. 3, p. e1355, 2020.

[6] S. B. Kotsiantis, C. J. Pierrakeas, and P. E. Pintelas, "Preventing student dropout in distance education using machine learning techniques," in Knowledge-Based Intelligent Information and Engineering Systems, Berlin: Springer, 2003, pp. 267-274.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16), San Francisco, CA, 2016, pp. 785-794.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS 30), Long Beach, CA, 2017, pp. 4765-4774.

[9] F. Agrusti, M. Mezzini, and G. Bonavolontà, "Deep learning classification for student dropout prediction in higher education," Interactive Learning Environments, vol. 28, no. 6, pp. 700-717, 2020.

[10] J. L. Rastrollo-Guerrero, J. A. Gómez-Pulido, and A. Durán-Domínguez, "Analyzing and predicting students' performance by means of machine learning: A review," Applied Sciences, vol. 10, no. 3, p. 1042, 2020.

[11] M. A. Hearst, S. T. Dumais, E. Osman, J. Platt, and B. Scholkopf, "Support vector machines," IEEE Intelligent Systems and their Applications, vol. 13, no. 4, pp. 18-28, 1998.

[12] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.

[14] G. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1-3, 1950.

[15] M. Kuhn and K. Johnson, Applied Predictive Modeling. New York: Springer, 2013.

[16] Node.js Foundation, "Node.js v20 LTS Architecture and Event Loop Documentation," [Online]. Available: https://nodejs.org/docs/latest-v20.x/api/.

[17] S. Ramirez, "FastAPI: Modern, High-Performance Web Framework for Python," [Online]. Available: https://fastapi.tiangolo.com/.

[18] MongoDB Inc., "MongoDB 6.0 Manual: Document Indexing and Storage Architecture," [Online]. Available: https://www.mongodb.com/docs/v6.0/.

[19] n8n GmbH, "Fair-Code Workflow Automation Documentation and Webhook Specifications," [Online]. Available: https://docs.n8n.io/.

[20] Anna University Chennai, "Regulations 2021: Choice Based Credit System (CBCS) for Affiliated Autonomous Institutions," Anna University Official Gazette, Chennai, India, 2021.

APPENDIX

Appendix A: COMPLETE PROJECT FOLDER STRUCTURE

Listing: Ecosystem Directory Hierarchy

```
FSD/
|-- backend/
|   |-- config/ (db.js)
|   |-- controllers/ (adminController.js, authController.js, meetingRequestController.js,
|   |                 mentorNoteController.js, notificationController.js, studentController.js)
|   |-- middleware/ (authMiddleware.js, rateLimiter.js, serviceKeyAuth.js)
|   |-- models/ (Curriculum.js, Faculty.js, Marks.js, MeetingRequest.js, MentorNote.js,
|   |            NotificationLog.js, OtpChallenge.js, Student.js, User.js)
|   |-- routes/ (api.js)
|   |-- services/ (aiServiceClient.js, mailer.js, notificationDispatcher.js)
|   |-- tests/ (otpAuth.test.js, notificationDispatcher.test.js, meetingRequestLifecycle.test.js, ...)
|   |-- package.json
|   `-- server.js
|-- frontend/
|   |-- src/
|   |   |-- components/ (ChatbotWidget.jsx, Navbar.jsx, ParentAlertModal.jsx, Sidebar.jsx)
|   |   |-- context/ (AuthContext.jsx, SidebarContext.jsx)
|   |   |-- pages/ (admin/AdminDashboard.jsx, mentor/MentorDashboard.jsx,
|   |   |           mentor/StudentDetail.jsx, public/LoginPage.jsx, student/StudentDashboard.jsx)
|   |   |-- services/ (api.js)
|   |   |-- App.jsx
|   |   `-- main.jsx
|   |-- package.json
|   `-- vite.config.js
|-- n8n/
|   |-- insighterp-notify.workflow.json
|   `-- README.md
`-- scripts/ (demo_check.mjs, ui-guard.mjs)

ai-dropout-prediction/
|-- ml-service/
|   |-- app/ (chatbot.py, distress.py, helplines.py, main.py, predictor.py)
|   |-- tests/ (test_ml_endpoints.py)
|   `-- requirements.txt
`-- models/ (dropout_model.pkl, preprocessor.pkl, evaluation/metrics.json)
```

Appendix B: INSTALLATION AND SETUP COMMANDS

Listing: Terminal Execution Commands

```
# 1. Start Python FastAPI ML Microservice (Port 8001)
cd ai-dropout-prediction/ml-service
python -m venv venv
venv\Scripts\activate          # Windows (source venv/bin/activate on Linux)
pip install -r requirements.txt
uvicorn app.main:app --host 127.0.0.1 --port 8001 --reload

# 2. Start AI Gateway Backend (Port 5001)
cd ai-dropout-prediction/backend
npm install
npm start

# 3. Start Baymate Standalone UI (Port 5174)
cd ai-dropout-prediction/frontend
npm install
npm run dev -- --port 5174

# 4. Start ERP Express Gateway Backend (Port 5000)
cd FSD/backend
npm install
npm run dev

# 5. Start ERP Vite React Frontend (Port 5173)
cd FSD/frontend
npm install
npm run dev -- --port 5173

# 6. Run Full Automated Test Suites & UI Guard
cd FSD/backend && npm test
cd FSD/frontend && npm run guard && npm run build
```

Appendix C: ENVIRONMENT VARIABLE CONFIGURATION

Listing: Sanitized Backend Configuration Template

```
# FSD/backend/.env.example
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://127.0.0.1:27017/student_dropout_erp
JWT_SECRET=super_secret_jwt_key_for_hardening_2026
AI_SERVICE_BASE_URL=http://localhost:5001
SERVICE_KEY=internal_erp_ai_service_key_2026

# Two-Factor Authentication (OTP)
OTP_ENABLED=true
OTP_TTL_SECONDS=300
OTP_MAX_ATTEMPTS=5
OTP_RESEND_COOLDOWN_SECONDS=30
OTP_MAX_RESENDS=3

# Automated Communications & n8n
NOTIFY_TRANSPORT=n8n
N8N_WEBHOOK_URL=http://localhost:5678/webhook/insighterp-notify
N8N_WEBHOOK_SECRET=n8n_insighterp_secret_token_2026
DEMO_SHARED_EMAIL=grouppresentation785@gmail.com
EMAIL_DRY_RUN=false
SMTP_HOST=smtp.gmail.com
SMTP_PORT=465
SMTP_USER=insighterp.notifications@gmail.com
SMTP_PASS=app_password_placeholder
```

Appendix D: IMPORTANT SOURCE CODE SNIPPETS

Listing: Cryptographic Constant-Time OTP Verification

```
// Excerpt: Timing-Safe OTP Code Verification (authController.js)
const secret = process.env.JWT_SECRET;
const submittedHash = crypto.createHmac('sha256', secret).update(String(code).trim()).digest('hex');

const submittedBuf = Buffer.from(submittedHash, 'hex');
const expectedBuf = Buffer.from(challenge.code_hash, 'hex');

if (submittedBuf.length !== expectedBuf.length || !crypto.timingSafeEqual(submittedBuf, expectedBuf)) {
  await OtpChallengeStore.incrementAttempts(challenge_id);
  const remaining = maxAttempts - (challenge.attempts + 1);
  if (remaining <= 0) {
    return res.status(429).json({ error: 'Too Many Requests', message: 'Challenge locked due to excessive
failed attempts' });
  }
  return res.status(401).json({ error: 'Unauthorized', message: `Invalid verification code. ${remaining}
attempts remaining.` });
}
```

Appendix E: API REQUEST AND RESPONSE EXAMPLES

Listing: Parent Alert REST Request and Response

```
// 1. Parent Alert Request (POST /api/students/:id/parent-alert)
Headers: Authorization: Bearer <mentor_jwt_token>, Content-Type: application/json
Body:
{
  "subject": "Academic Attendance Notice: Urgent Mentor Advisory",
  "message": "Dear Parent, Bhoomika's attendance has dropped to 62.8% with 27 active arrears."
}

// 2. Parent Alert Response (200 OK)
{
  "success": true,
  "message": "Parent alert dispatched successfully",
  "data": {
    "event_id": "evt-7c9b2f1e-45a8-48b2-9d33-872f1a91e502",
    "status": "sent",
    "channel": "n8n",
    "recipient": "parent.bhoomika@gmail.com"
  }
}
```

Appendix F: DATABASE SCHEMA AND MODEL DETAILS

Mongoose models define strict validation, defaults, and compound indexes. The OtpChallenge model includes an automatic TTL index on 'expires_at' (expireAfterSeconds: 0), ensuring automatic document purging by MongoDB's background thread.

Appendix G: ADDITIONAL TEST CASES

Supplementary regression suites verify that curriculum queries filter correctly by branch and semester, distinct branch lists sort alphabetically, and student mark records aggregate accurately across all 8 academic semesters.

Appendix H: ADDITIONAL PROJECT SCREENSHOTS

Supplementary screenshot slots for auxiliary modules (Curriculum browser, Branch explorer, Institution directory, and Terminal test execution logs).